

Utrecht University Repository

Title	XNLP-Completeness for Parameterized Problems on Graphs with a Linear Structure
Authors	Bodlaender, Hans L.; Groenland, Carla; Jacob, Hugo; Jaffke, Lars; Lima, Paloma T.
Published in	Algorithmica
Publication Date	2025-04
Link	https://dspace.library.uu.nl/handle/1874/473934
Citation	Bodlaender, H L, Groenland, C, Jacob, H, Jaffke, L & Lima, P T 2025, 'XNLP-Completeness for Parameterized Problems on Graphs with a Linear Structure', Algorithmica, vol. 87, pp. 465-506. https://doi.org/10.1007/s00453-024-01274-9
Versions / License	Publisher version
Rights	https://www.uu.nl/en/university-library/license-and-reuse-conditions



XNLP-Completeness for Parameterized Problems on Graphs with a Linear Structure

Hans L. Bodlaender¹ · Carla Groenland¹ · Hugo Jacob² · Lars Jaffke^{3,4} · Paloma T. Lima⁵

Received: 22 December 2022 / Accepted: 20 September 2024 / Published online: 4 November 2024
© The Author(s) 2024

Abstract

In this paper, we showcase the class XNLP as a natural place for many hard problems parameterized by linear width measures. This strengthens existing $W[1]$ -hardness proofs for these problems, since XNLP-hardness implies $W[t]$ -hardness for all t . It also indicates, via a conjecture by Pilipczuk and Wrochna (ACM Trans Comput Theory 9:1–36, 2018), that any XP algorithm for such problems is likely to require XP space. In particular, we show XNLP-completeness for natural problems parameterized by pathwidth, linear clique-width, and linear mim-width. The problems we consider are INDEPENDENT SET, DOMINATING SET, ODD CYCLE TRANSVERSAL, $(q-)$ COLORING, MAX CUT, MAXIMUM REGULAR INDUCED SUBGRAPH, FEEDBACK VERTEX SET, CAPACITATED (RED- BLUE) DOMINATING SET, CAPACITATED VERTEX COVER and BIPARTITE BANDWIDTH.

An extended abstract of this work appeared at IPEC 2022 [3].

✉ Lars Jaffke
lars.jaffke@nhh.no
Hans L. Bodlaender
h.l.bodlaender@uu.nl
Carla Groenland
c.e.groenland@tudelft.nl
Hugo Jacob
hugo.jacob@lirmm.fr
Paloma T. Lima
palt@itu.dk

- ¹ Department of Information and Computing Sciences, Utrecht University, Utrecht, The Netherlands
- ² CNRS, LIRMM, Université de Montpellier, Montpellier, France
- ³ Department of Business and Management Science, NHH Norwegian School of Economics, Bergen, Norway
- ⁴ Department of Informatics, University of Bergen, Bergen, Norway
- ⁵ Department of Computer Science, IT University of Copenhagen, Copenhagen, Denmark

Keywords Parameterized complexity · XNLP · Linear clique-width · Pathwidth · Linear mim-width · Bandwidth

1 Introduction

Since the inception of parameterized complexity in the late 1980s and early 1990s, much research has been done on establishing the complexity of parameterized problems. Typically one is particularly interested in either designing FPT-algorithms for these problems, or to prove them $W[t]$ -hard, for some t , which provides evidence that such a problem is not likely to be fixed-parameter tractable. As opposed to the classical P versus NP-complete setting, the question of membership in some class of the W -hierarchy is often much less clear. While some natural problems such as INDEPENDENT SET and DOMINATING SET are known to be $W[1]$ -complete and $W[2]$ -complete, respectively, many other problems are unknown to be complete for a class of parameterized problems, and even conjectured not to be in the W -hierarchy. Recently, building upon work by Elberfeld et al. [13], Bodlaender et al. [4] introduced a complexity class called XNLP, which gives a way of addressing this question.

The class XNLP consists of the parameterized problems that can be solved with a non-deterministic algorithm that uses $f(k) \log n$ space and $f(k)n^c$ time, where f is a computable function, n is the input size, k is the parameter and c is a constant. In particular, XNLP-hardness implies $W[t]$ -hardness for all t . Therefore it is unlikely that any XNLP-hard problem is complete for some $W[t]$.

One success story within parameterized algorithms and complexity is the use of width measures of graphs as parameters (see, e.g., [10]). Typically, such width measures are defined in terms of a tree-like decomposition of a graph, and the width describes the complexity of the decomposition, and therefore, in turn, of the graph. Such width measures also have linear variants, where the decomposition resembles a path instead of a tree. In this work, we provide evidence that the class XNLP is the ‘natural home’ for hard problems parameterized by linear width measures.

Let us give some intuitive explanation why this is the case. A typical dynamic programming algorithm that uses such a linear decomposition stores, at each node of the path, some partial solutions associated with it. The table entries associated with the nodes are then filled in the order in which they appear on the path. If one turns such an algorithm into a nondeterministic algorithm, it often suffices at the i -th node to nondeterministically determine the table index corresponding to the correct partial solution (if it exists) from the table entry that was previously determined for the $(i - 1)$ -th node. In such a case, membership in XNLP follows if each single table entry of such a DP algorithm can be represented by $f(k) \log n$ bits (where k is the width) and if the nondeterministic step does not require a computation that uses significantly more space. This is often the case. Now, such an approach fails for tree-like decompositions, since even a nondeterministic algorithm might have to keep too many table entries at some point during the computation. One common situation in which this occurs is when the algorithm needs to store one table entry for each level of the decomposition. This incurs a multiplicative factor in the memory usage that depends on the height of the tree, which can be prohibitively large.

In this direction, Bodlaender et al. [4] showed that LIST COLOURING parameterized by the pathwidth of the input graph, and BANDWIDTH are XNLP-complete. In this paper, we show XNLP-completeness of fundamental graph problems parameterized by linear variants of well-established width measures, such as pathwidth, linear clique-width and linear mim-width, as well as some of their logarithmic analogues.

Besides showing $W[t]$ -hardness for all t , XNLP-hardness also provides insight into the space complexity of parameterized problems. Pilipczuk and Wrochna [26] proposed the following conjecture.¹

Conjecture 1 (Slice-wise Polynomial Space Conjecture [26]) *XNLP-hard problems do not have an algorithm that runs in $n^{f(k)}$ time and $f(k)n^c$ space, with f a computable function, k the parameter, n the input size, and c a constant.*

Typically, membership in XP for the problems studied in our paper follows from a dynamic programming approach that uses an “XP amount” of memory. In fact, the run times of these algorithms can often be expressed as some low-degree polynomial of their space usage. XNLP-hardness indicates (via Conjecture 1) that such dynamic programming algorithms are in some sense ‘optimal’ as it rules out the existence of XP-algorithms that require only an “FPT amount” of space.

Linear width measures and logarithmic analogues. The width measures we consider in this work include linear variants of arguably the most prominent measures, and some of their generalizations. Pathwidth is a linear variant of the classic treewidth parameter, which, informally speaking, measures how close a connected graph is to being a tree. In this vein, pathwidth measures how close a connected graph is to being a path. Clique-width (or, equivalently, rank-width) generalizes treewidth to several simply structured dense graphs, and its linear counterpart is called linear clique-width (linear rank-width). Maximum induced matching width [27], or mim-width for short, in turn generalizes clique-width and remains bounded even on well-studied graph classes such as interval and permutation graphs, where the clique-width is known to be unbounded. In fact, for most of these classes the *linear* mim-width is bounded.

We also introduce a new parameter that we call logarithmic linear clique-width, analogous to the parameter logarithmic pathwidth that was introduced by Bodlaender et al. [4]. For an n -vertex graph of linear clique-width k , logarithmic linear clique-width takes the value $\lceil k / \log n \rceil$. We stress the fact that XNLP-hardness parameterized by a logarithmic parameter implies that there is no algorithm solving the problem in time $2^{O(k)} n^{O(1)}$ and space $k^{O(1)} n^{O(1)}$, where k is the original parameter,² under Conjecture 1. Such results can complement existing (S)ETH lower bounds for single exponential FPT algorithms with lower bounds on the space requirements of such algorithms.

Bipartite bandwidth. Finally, we consider a bipartite variant of the notoriously difficult [6] problem of computing the bandwidth of a graph. Here, for a bipartite graph with vertex bipartition (A, B) and bandwidth target value w , we seek an ordering α of A and an ordering β of B , such that for each edge ab , $|\alpha(a) - \beta(b)| \leq w$. We consider

¹ The statement of the conjecture here is equivalent to the conjecture on time and memory use for the LONGEST COMMON SUBSEQUENCE problem from [26]; the name of the conjecture is taken as analogue to the naming of XP as problems that use *slice-wise polynomial time* (see [10, Section 1.1]).

² Indeed, replacing k with $k' \log n$, this gives running time $2^{O(k' \log n)} = n^{O(k')}$ and space $k'^{O(1)} n^{O(1)}$, which is excluded by the conjecture.

this problem parameterized by w , and show that it is XNLP-complete, even when the input graph is a tree.

Our results. We summarize our results in the following theorem.

Theorem 1 *The following problems are XNLP-complete.*

1. CAPACITATED RED-BLUE DOMINATING SET, CAPACITATED DOMINATING SET and CAPACITATED VERTEX COVER parameterized by pathwidth.
2. COLORING, MAXIMUM REGULAR INDUCED SUBGRAPH, and MAX CUT parameterized by linear clique-width.
3. q -COLORING and ODD CYCLE TRANSVERSAL parameterized by logarithmic pathwidth or logarithmic linear clique-width.
4. INDEPENDENT SET, DOMINATING SET, FEEDBACK VERTEX SET, and q -COLORING for fixed $q \geq 5$ parameterized by linear mim-width.
5. BIPARTITE BANDWIDTH, even if the input graph is a tree.

Furthermore, FEEDBACK VERTEX SET parameterized by logarithmic pathwidth or logarithmic linear clique-width is XNLP-hard.

Note that Theorem 1(2) and (4) include the first XNLP-completeness results for graph problems with the linear clique-width and linear mim-width as parameter.

Related Work. Guillemot [18] introduced the class WNL (which equals XNLP closed under fpt-reductions), and showed some problems to be complete for WNL, including a version of LONGEST COMMON SUBSEQUENCE. The class XNLP (under the name $N[f \text{ poly}, f \log]$) was introduced by Elberfeld et al. [13], who also showed a number of problems, including LINEAR CELLULAR AUTOMATON ACCEPTANCE, to be complete for the class. A large number of parameterized problems was shown to be XNLP-complete recently by Bodlaender et al. [4]. Recently, in work that aims at separating the complexity of treewidth and pathwidth at one side, and stable gonality at another side, Bodlaender et al. [1] showed a number of flow problems parameterized by pathwidth to be complete for XNLP. Very recently, a variant of XNLP targeted at tree-structured problems was introduced in [2].

2 Overview of the Results

In this section, we give a bird's-eye view of the results proved in this paper, and discuss related work for the specific problems we consider.

Parameterized by linear clique-width. We consider the MAX CUT, the COLORING, and the MAXIMUM REGULAR INDUCED SUBGRAPH problems parameterized by linear clique-width. Let $E(V_1, V_2)$ denote the set of edges with one endpoint in V_1 and one endpoint in V_2 . (Note that linear k -expressions, which witness that a graph has linear clique-width at most k , are defined in Sect. 3.3.)

MAX CUT / linear clique-width

Input: A graph $G = (V, E)$ described by a given linear k -expression describing G and an integer W .

Parameter: k .

Question: Is there a bipartition of V into (V_1, V_2) such that $|E(V_1, V_2)| \geq W$?

In 1994, Wanke [28] showed that MAX CUT is in XP for graphs of bounded NLC-width, which directly implies XP-membership with clique-width as parameter, as NLC-width and clique-width are linearly related [21]. In 2014, Fomin et al. [15] considered the fine grained complexity for MAX CUT for graphs of small clique-width, giving an algorithm with improved running time and showing asymptotic optimality (assuming the Exponential Time Hypothesis). From their results, it follows that MAX CUT is $W[1]$ -hard with clique-width as parameter. In Sect. 4.1, we prove the following theorem.

Theorem 2 MAX CUT with linear clique-width as parameter is XNLP-complete.

Next, we consider the classical COLORING problem which, given a graph G and an integer ℓ , asks if G has a proper coloring with ℓ colors. Similarly to the story of the MAX CUT problem, COLORING parameterized by clique-width was shown to be in XP by Wanke [28], and a $W[1]$ -hardness proof only followed in 2010 by Fomin et al. [14]. The XP algorithm for coloring runs in time $n^{O(2^k)}$, where k is the clique-width, and Fomin et al. [16] even showed that this run time can probably not be substantially improved: an algorithm running in time $n^{2^{o(k)}}$ would refute the ETH. We prove the following in Sect. 4.1.

Theorem 3 COLORING parameterized by linear clique-width is XNLP-complete.

Lastly, we consider the MAXIMUM REGULAR INDUCED SUBGRAPH problem. The problem was studied by several authors, including who show among others an algorithm that uses linear time for graphs of bounded treewidth, where the time depends single exponentially on the treewidth. Moser and Thilikos [25], and independently Mathieson and Szeider [24] show (among other results) that the problem is $W[1]$ -hard when the size of the subgraph (parameter W in our description below) is used as parameter. Broersma et al. [8] give XP algorithms for several problems, including MAXIMUM REGULAR INDUCED SUBGRAPH for graphs of bounded clique-width. (Recall that linear k -expressions are defined in Sect. 3.3.)

MAXIMUM REGULAR INDUCED SUBGRAPH / linear clique-width

Input: A graph G described by a given linear k -expression and two integers W and D .

Parameter: k .

Question: Is there a D -regular induced subgraph of G on at least W vertices?

We show the following in Sect. 4.1.

Theorem 4 MAXIMUM REGULAR INDUCED SUBGRAPH parameterized by linear clique-width is XNLP-complete.

Parameterized by pathwidth. We consider the CAPACITATED RED- BLUE DOMINATING SET and CAPACITATED DOMINATING SET problems. Below, we give the formal statement of the problems, where we have the width of the path decomposition as parameter. One of the reasons of interest in these problems is that they model facility location problems: the red vertices model possible facilities that can serve a bounded number of clients which are modelled by the blue vertices. In the following definitions, for sets X, Y , a function $f: X \rightarrow Y$ and $y \in Y$, with slight abuse of notation, we denote by $f^{-1}(y)$ the subset of X that maps to y .

CAPACITATED RED- BLUE DOMINATING SET / pathwidth

Input: A bipartite graph $G = (R, B, E)$, a path decomposition of G of width ℓ , a capacity function $c: R \rightarrow \mathbb{N}$, and an integer k .

Parameter: ℓ .

Question: Is there a subset S of R of size k , and an assignment of blue vertices $f: B \rightarrow S$ such that $\{w, f(w)\} \in E$ for all $w \in B$ and $|f^{-1}(v)| \leq c(v)$ for all v in S ?

CAPACITATED DOMINATING SET / pathwidth

Input: A graph $G = (V, E)$, a path decomposition of G of width ℓ , a capacity function $c: V \rightarrow \mathbb{N}$, and an integer k .

Parameter: ℓ .

Question: Is there a subset S of V of size k , and an assignment of the vertices $f: V \rightarrow S$ such that $\{w, f(w)\} \in E$ or $w = f(w)$ for all $w \in V$ and $|f^{-1}(v)| \leq c(v)$ for all v in S ?

CAPACITATED VERTEX COVER / pathwidth

Input: A graph $G = (V, E)$, a path decomposition of G of width ℓ , a capacity function $c: V \rightarrow \mathbb{N}$, and an integer k .

Parameter: ℓ .

Question: Is there a subset S of V of size k , and an assignment of the edges $f: E \rightarrow S$ such that $f(e)$ is an endpoint of e for all $e \in E$ and $|f^{-1}(v)| \leq c(v)$ for all v in S ?

In 2008, Dom et al. [11] showed that CAPACITATED DOMINATING SET is $W[1]$ -hard, with the treewidth and solution size k as combined parameter, and CAPACITATED VERTEX COVER with treewidth as parameter is $W[1]$ -hard. CAPACITATED DOMINATING SET was shown to be $W[1]$ -hard for planar graphs, with the solution size as parameter by Bodlaender et al. [5]. Fomin et al. [15] give bounds for the fine grained complexity of CAPACITATED RED- BLUE DOMINATING SET, for graphs with a small feedback vertex set; their results imply that the problem is $W[1]$ -hard with feedback vertex set as parameter. The proof of the following theorem can be found in Sect. 4.2. A variant of the proof gives the same complexity result for CAPACITATED VERTEX COVER.

Theorem 5 CAPACITATED RED- BLUE DOMINATING SET and CAPACITATED DOMINATING SET parameterized by pathwidth are $XNLP$ -complete.

Parameterized by logarithmic linear clique-width. Bodlaender et al. [4] introduced the parameter *logarithmic pathwidth* as $\text{pw}/\log n$ for an n -vertex graph of pathwidth pw . This allows the pathwidth to be linear in the logarithm of the number of vertices of the graph. Here we introduce the *logarithmic linear clique-width* as $\text{lcw}/\log n$ for graphs on n vertices with linear clique-width lcw .

We provide new XNLP-complete problems for the parameter logarithmic pathwidth, and show that these problems and the previously known XNLP-complete problems for this parameter [4] are also complete for the parameter logarithmic linear clique-width. Our results are summarized below.

The motivation to study the logarithmic linear clique-width or logarithmic pathwidth comes from the observation that many FPT algorithms with linear clique-width or pathwidth as parameter have a single exponential time dependency on the parameter. Thus, if linear clique-width or pathwidth is logarithmic in the size of the graph, these algorithms turn into XP algorithms. We show the following result in Sect. 4.3.

Theorem 6 *When parameterized by logarithmic pathwidth or logarithmic linear clique-width, INDEPENDENT SET, DOMINATING SET, q -LIST-COLORING for $q > 2$, and ODD CYCLE TRANSVERSAL are XNLP-complete, and FEEDBACK VERTEX SET is XNLP-hard.*

Lokshtanov et al. [23] established (tight) lower bounds for these problems for the parameter pathwidth under the Strong Exponential Time Hypothesis. Several of our gadgets are based on those used for these lower bounds by [23].

Parameterized by linear mim-width. We prove that several fundamental graph problems are XNLP-complete when parameterized by the mim-width of a given linear order of the input graph. $W[1]$ -hardness for INDEPENDENT SET and DOMINATING SET in this parameterization was shown by Fomin et al. [17], and for FEEDBACK VERTEX SET by Jaffke et al. [20]. For q -COLORING, $W[1]$ -hardness was not known before our work. We would like to point out that our XNLP-hardness proof uses a gadget that requires five colors to construct, and it would be interesting to see if this can be improved to three colors. In Sect. 4.4 we prove the result below.

Theorem 7 *When parameterized by linear mim-width, INDEPENDENT SET, DOMINATING SET, q -COLORING for any fixed $q \geq 5$ and FEEDBACK VERTEX SET are XNLP-complete.*

Bipartite bandwidth. We consider the following bipartite variant of the BANDWIDTH problem.

BIPARTITE BANDWIDTH

Input: A bipartite graph $G = (X, Y, E)$ and an integer k .

Parameter: k .

Question: Are there orderings $\alpha : X \rightarrow [n]$ and $\beta : Y \rightarrow [m]$ such that for each $uv \in E$, $|\alpha(u) - \beta(v)| \leq k$?

A possible application of this problem is as follows. Let a matrix M be given. Create a vertex $x_i \in X$ for each row i and a vertex $y_j \in Y$ for each column j , and let x_i be adjacent to y_j if and only if $M_{i,j} \neq 0$. This graph has bipartite bandwidth at most k if

and only if the rows and columns of M can be permuted (individually) in such a way that all non-zero entries are within k distance from the main diagonal. We show the following in Sect. 4.5.

Theorem 8 BIPARTITE BANDWIDTH is XNLP-complete for trees.

3 Preliminaries

The required background on the computational problems studied in this paper is given in their respective sections. The notions relevant to the entire paper are defined below.

We write $[n] = \{1, \dots, n\}$ and $[a, b]$ for the set of integers x with $a \leq x \leq b$. All logarithms in this paper have base 2. We use $\mathbb{N}$ for the set of the natural numbers $\{0, 1, 2, \dots\}$, and $\mathbb{Z}^+$ denotes the set of the positive natural numbers $\{1, 2, \dots\}$. We write $N(S)$ and $N[S] = N(S) \cup S$ for the open and closed neighborhood of S .

3.1 Definition of the Class XNLP

In this paper, we study parameterized decision problems, which are subsets of $\Sigma^* \times \mathbb{N}$, for a finite alphabet Σ . We assume the reader to be familiar with notions from parameterized complexity, such as XP, $W[1]$, $W[2]$, ..., $W[P]$ (see e.g. [12]). The class XNLP (denoted $N[f \text{ poly}, f \log]$ by [13]) consists of the parameterized decision problems that can be solved by a non-deterministic algorithm that simultaneously uses at most $f(k)n^c$ time and at most $f(k) \log n$ space, on an input (x, k) , where x can be denoted with n bits, f a computable function, and c a constant. We assume that functions f of the parameter in time and resource bounds are computable — this is called *strongly uniform* by Downey and Fellows [12]. More information about the complexity class XNLP can be found in [4].

3.2 Reductions

In the remainder of the paper, unless stated otherwise, completeness for XNLP is with respect to pl-reductions, which are defined below. The definitions are based upon the formulations in [13]. A *parameterized reduction* from a parameterized problem $Q_1 \subseteq \Sigma_1^* \times \mathbb{N}$ to a parameterized problem $Q_2 \subseteq \Sigma_2^* \times \mathbb{N}$ is a function $f : \Sigma_1^* \times \mathbb{N} \rightarrow \Sigma_2^* \times \mathbb{N}$, such that the following holds.

1. For all $(x, k) \in \Sigma_1^* \times \mathbb{N}$, $(x, k) \in Q_1$ if and only if $f((x, k)) \in Q_2$.
2. There is a computable function g , such that for all $(x, k) \in \Sigma_1^* \times \mathbb{N}$, if $f((x, k)) = (y, k')$, then $k' \leq g(k)$.

A *parameterized logspace reduction* or *pl-reduction* is a parameterized reduction for which there is an algorithm that given any $(x, k) \in \Sigma_1^* \times \mathbb{N}$ computes $f((x, k))$ in space $O(g(k) + \log n)$, with g a computable function and $n = |x|$ the number of bits to denote x .

3.3 Pathwidth, Linear Clique-Width, and Linear Mim-Width

A *path decomposition* of a graph $G = (V, E)$ is a sequence $(X_1, X_2, \dots, X_r)$ of subsets of V with the following properties.

1. $\bigcup_{1 \leq i \leq r} X_i = V$.
2. For all $\{v, w\} \in E$, there is an $i \in I$ with $v, w \in X_i$.
3. For all $1 \leq i_0 < i_1 < i_2 \leq r$, $X_{i_0} \cap X_{i_2} \subseteq X_{i_1}$.

The *width* of a path decomposition $(X_1, X_2, \dots, X_r)$ equals $\max_{1 \leq i \leq r} |X_i| - 1$, and the *pathwidth* pw of a graph G is the minimum width of a path decomposition of G .

A *k-labeled graph* is a graph $G = (V, E)$ together with a labeling function $\Gamma : V \rightarrow [k]$. A *k-expression* constructs a *k-labeled graph* by the means of the following operations:

1. *Vertex creation*: $i(v)$ is the *k-labeled graph* consisting of a single vertex v which is assigned label i .
2. *Disjoint union*: $H \oplus G$ is the disjoint union of *k-labeled graphs* H and G .
3. *Join*: $\eta_{i \times j}(G)$ is the *k-labeled graph* obtained by adding all possible edges between vertices with label i and vertices with label j to G .
4. *Renaming label*: $\rho_{i \rightarrow j}(G)$ is the *k-labeled graph* obtained by assigning label j to all vertices labelled i in G .

A *linear k-expression* is a *k-expression* with the additional condition that one of the arguments of the disjoint union operation needs to be a graph consisting of a single vertex. The *clique-width* $\text{cw}(G)$ (resp. linear clique-width $\text{lcw}(G)$) of a graph G is the minimal k such that G can be constructed by a *k-expression* (resp. linear *k-expression*) with any labeling.

For a graph $G = (V, E)$ and $A, B \subseteq V$ with $A \cap B = \emptyset$, we let $G[A, B]$ be the bipartite subgraph of G with vertices $A \cup B$ and edges $\{ab \mid ab \in E, a \in A, b \in B\}$. We let $\text{cutmim}_G(A, B)$ be the size of a maximum induced matching in $G[A, B]$ and $\text{mim}_G(A) = \text{cutmim}_G(A, V \setminus A)$. Here, an induced matching $M \subseteq E$ is a matching such that there are no additional edges between the endpoints of M in the graph in question. The *mim-width* of a linear order $v_1, \dots, v_n$ of V is the maximum, over all i , of $\text{mim}_G(\{v_1, \dots, v_i\})$. The *linear mim-width* of G is the minimum *mim-width* over all linear orders of V .

3.4 Chained variants of SATISFIABILITY and MULTICOLORED CLIQUE

In [4], the following problems were introduced, and shown to be XNLP-complete.

CHAINED POSITIVE CNF- SAT

Input: r sets of Boolean variables $X_1, X_2, \dots, X_r$, each of size q ; an integer $k \in \mathbb{N}$; Boolean formula ϕ , which is in conjunctive normal form and an expression on $2q$ variables, using only positive literals; for each i , a partition of X_i into $X_{i,1}, \dots, X_{i,k}$ such that $\forall j, j' \in [k], |X_{i,j}| = |X_{i,j'}|$.

Parameter: k .

Question: Is it possible to satisfy the formula

$$\bigwedge_{1 \leq i \leq r-1} \phi(X_i, X_{i+1})$$

by setting from each set $X_{i,j}$ exactly 1 variable to true and all others to false?

CHAINED MULTICOLORED CLIQUE

Input: Graph $G = (V, E)$, partition of V into $V_1, \dots, V_r$, such that for each edge $uv \in E$ with $u \in V_i$ and $v \in V_j$, $|i - j| \leq 1$, function $f: V \rightarrow [k]$.

Parameter: k .

Question: Is there a set $W \subseteq V$ such that for all $i \in [r - 1]$, $W \cap (V_i \cup V_{i+1})$ is a clique, and for each $i \in [r]$ and $j \in [k]$, there is a vertex $v \in W \cap V_i$ with $f(v) = j$?

The CHAINED MULTICOLORED INDEPENDENT SET problem is defined analogously, with the only difference that the solution W is required to be an independent set.

Theorem 9 (Bodlaender et al. [4]) CHAINED POSITIVE CNF- SAT, CHAINED MULTICOLORED CLIQUE and CHAINED MULTICOLORED INDEPENDENT SET are XNLP-complete.

4 Problems Parameterized by Linear Width Measures

In this section we prove XNLP-completeness for three of the problems mentioned in Sect. 2 parameterized by linear width measures.

4.1 Problems Parameterized by Linear Clique-Width

In this section, we consider various problems with the linear clique-width as parameter, and show them to be XNLP-complete.

We first consider MAX CUT. Our result is based upon the XNLP-hardness result for a problem, called CIRCULATING ORIENTATION, with pathwidth as parameter. Borrowing from terminology from flows in graphs, we say that a directed graph $G = (V, A)$ with for each arc $a \in A$ a weight $w(a) \in \mathbb{N}$, is a *circulation*, if for each vertex v , the total weight of all incoming arcs at v equals the total weight of all arcs outgoing from v . We reduce from the following problem.

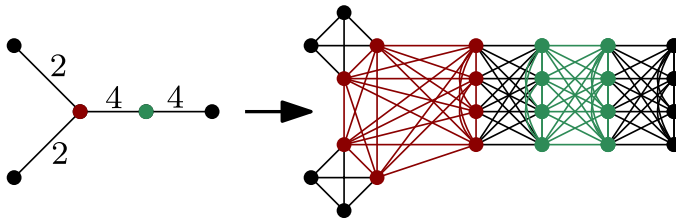


Fig. 1 Example for the construction of the hardness proof of MAX CUT (fragment)

CIRCULATING ORIENTATION / pathwidth

Input: An undirected graph $G = (V, E)$ with a path decomposition of G of width ℓ , an edge weight function $w : E \rightarrow \mathbb{N}$, given in unary notation.

Parameter: ℓ .

Question: Is there an orientation of G that is a circulation?

Theorem 10 (Bodlaender et al. [1]) CIRCULATING ORIENTATION is XNLP-complete.

Theorem 2 MAX CUT with linear clique-width as parameter is XNLP-complete.

Proof We first show membership in XNLP. The main idea is to turn the existing dynamic programming that solves the problem given a k -expression of an n -vertex graph of linear clique-width k into a non-deterministic algorithm, by guessing an element from a table instead of building full tables. For each vertex creation, we guess on which side of the partition the vertex is. We maintain the following certificate: for each label, the number of vertices on each side of the bipartition, and the number of edges of the current expression that were in the cut. Since there are at most k labels and the size of the cut is bounded by the number of edges, this certificate uses only $O(k \log n)$ bits.

To show hardness for XNLP, we reduce from CIRCULATING ORIENTATION with pathwidth as parameter. Suppose we have an instance for CIRCULATING ORIENTATION: an undirected graph G with edge weight function w . For each vertex v , write $D(v)$ as the total weight of all edges incident to v .

We build a new, undirected graph $H = (V_H, E_H)$ as follows. For each vertex $v \in V$, each edge e with v as one of its endpoints, and each integer $i \in [1, w(e)]$, we create a vertex $x_{v,e,i}$. Two distinct vertices $x_{v,e,i}$ and $x_{w,e',j}$ are adjacent if and only if $v = w$ or $e = e'$. In other words: for each vertex $v \in V$, we have a clique with $D(v)$ vertices, which consists of all vertices of the form $x_{v,\cdot,\cdot}$, that we call *the clique of v* . For each edge $e = \{v, w\} \in E$, we have a clique with $2w(e)$ vertices, namely all vertices of the form $x_{v,e,\cdot}$ and $x_{w,e,\cdot}$. See Fig. 1 for a partial example.

Claim 1 G has a circulating orientation if and only if H has a bipartition that cuts $\sum_{e \in E} w(e)^2 + \sum_{v \in V} D(v)^2/4$ edges.

Proof Suppose G has a circulating orientation. For each edge $e = \{v, w\}$, if the orientation directs v to w , then add all vertices of the form $x_{v,e,i}$ to Z_1 and all vertices

of the form $x_{w,e,i}$ to Z_2 ($i \in [1, w(e)]$); otherwise, add all vertices of the form $x_{v,e,i}$ to Z_2 and all vertices of the form $x_{w,e,i}$ to Z_1 ($i \in [1, w(e)]$).

Since we started from a circulating orientation, for each vertex v there are $D(v)/2 \times D(v)/2$ edges of the form $\{x_{v,\cdot,\cdot}, x_{v,\cdot,\cdot}\}$ crossing the bipartition. Moreover, there are $w(e) \times w(e)$ edges of the form $\{x_{v,e,\cdot}, x_{w,e,\cdot}\}$ crossing the bipartition for each edge $e = \{v, w\}$. We conclude that the bipartition cuts the required number of edges.

Now, suppose we have a partition Z_1, Z_2 of V_H with $\sum_{e \in E} w(e)^2 + \sum_{v \in V} D(v)^2/4$ edges between Z_1 and Z_2 . We distinguish two types of edges in $E_H \cap (Z_1 \times Z_2)$. A Type 1 edge is an edge between two vertices $x_{v,e,i}$ and $x_{v,e',j}$ (i.e., it is in the clique of a vertex v). A Type 2 edge is an edge between two vertices $x_{v,e,i}$ and $x_{w,e,j}$ for some edge $e = \{v, w\}$. Note that each edge in H is of Type 1 or Type 2 and that H has precisely $\sum_{e \in E} w(e)^2$ Type 2 edges.

For each vertex $v \in V$, we consider how many Type 1 edges (those in the clique of v) are in $Z_1 \times Z_2$. If we have α vertices in the clique of v that belong to Z_1 , then $D(v) - \alpha$ vertices in the clique of v belong to Z_2 , and thus, in this clique, we cut $\alpha \cdot (D(v) - \alpha) \leq D(v)^2/4$ edges; the maximum possible is reached when $\alpha = D(v)/2$.

It follows that the number of Type 1 edges that are cut is at most $\sum_{v \in V} D(v)^2/4$. So, we must cut all Type 2 edges, i.e., for each edge $e = \{v, w\}$, all edges of the form $\{x_{v,e,i}, x_{w,e,j}\}$ are between a vertex in Z_1 and a vertex in Z_2 . It follows that we either have that all vertices of the form $x_{v,e,i}$ are in Z_1 and all vertices of the form $x_{w,e,i}$ are in Z_2 — in which case we direct the edge e from v to w ; or all vertices of the form $x_{v,e,i}$ are in Z_2 and all vertices of the form $x_{w,e,i}$ are in Z_1 , and now we direct the edge from w to v .

For each vertex $v \in V$, we must have exactly $D(v)/2$ vertices from the clique of v in Z_1 and equally many vertices in Z_2 ; otherwise, we cannot reach the required number of cut edges. Now, the total weight of all edges that we directed out of v precisely equals the number of vertices in the clique of v in Z_1 , and similarly, the total weight of all edges that we directed towards v precisely equals the number of vertices in the clique of v in Z_2 . Both numbers equal $D(v)/2$. As this holds for each $v \in V$, the orientation defined above is a circulation. $\triangleleft$

Finally, we show that we can construct a linear clique expression for H given a path decomposition of G ; the number of colors we use for the clique width construction equals the width of the path decomposition plus 4. The construction uses ideas for constructing clique width constructions for line graphs of graphs of bounded treewidth; see [19].

Suppose we have a nice path decomposition, which uses introduce vertex, introduce edge, and forget nodes. We use $k + 1$ active colors — each active color will correspond to one vertex in the current bag. We also have an inactive color, which we will denote by the letter o . We also use two temporary colors, which we call α and β .

We sequentially visit the bags of the path decomposition. Bags correspond to a number of steps of the construction of H , as described next. If we introduce a vertex, we select a currently unused active color, and say this is the color of that vertex, and assume it to be used. If we introduce an edge $e = \{v, w\}$, we add the vertices $x_{v,e,i}$ one by one, each with the color α . Then, we add the vertices $x_{w,e,i}$ one by one, each with the color β . Now, we add all edges between vertices of color α and

β . Now, recolor all vertices of color α by the color of v . Then, recolor the vertices of color β by the color of w . If we forget a vertex v , we first add edges between all vertices of the color of v — at this point, these are all vertices in the clique of v , thus effectively ensuring that this set of vertices indeed is a clique. Then, recolor the vertices with the color of v with the inactive color o . Consider the color of v now unused.

One can verify that this indeed constructs precisely H , and that the construction can be done with $f(k) \log n$ additional space. $\square$

We consider the COLORING problem parameterized by linear clique-width next. To prove XNLP-hardness, we reduce from the following problem, which was shown to be XNLP-complete by Bodlaender et al. [1].

MINIMUM MAXIMUM OUTDEGREE / pathwidth

Input: Undirected weighted graph $G = (V, E, w)$ with weight function $w: E \rightarrow \mathbb{Z}^+$ given in unary notation, integer r , a path decomposition of G of width ℓ .

Parameter: ℓ .

Question: Is there an orientation of G such that for each $v \in V$, the total weight of all edges directed out of v is at most r ?

Lemma 1 COLORING parameterized by linear clique-width is in XNLP.

Proof The proof follows the lines of the XP algorithm of [22]. We keep the following certificate in memory: for each nonempty subset S of the k labels, we store the number of colors that appear exactly in this subset of labels. This requires $2^k \log n$ bits.

- For operation $\eta_{i \times j}$, we reject if there exists $S \supseteq \{i, j\}$ with at least one color appearing in exactly S .
- For operation $\rho_{i \rightarrow j}$, we simply update the values in our certificate using the inclusion exclusion principle.
- For the disjoint union with an isolated vertex v of label i , we nondeterministically choose if its color is already present in label i or not. In case it is, we do not update the certificate but reject if i was empty (i.e. there is no color that is exactly present in any subset of labels including i). In case it is not, we nondeterministically choose a subset S of labels that does not include i with a corresponding nonzero entry in the certificate, decrement its entry (if it is not the empty set), and then increment the entry of $S \cup \{i\}$.

We start from one of the deepest leaves of the expression and apply the previous computations until we reach its root. We then accept if the total over all entries of the certificate is at most the target value. The computation is done with $O(2^k \log n)$ bits, in FPT time. $\square$

The following gadget will be central in our hardness proof.

Lemma 2 For each positive integer α , there is a graph H_α with the following properties:

1. There is a linear 10-expression constructing H_α , constructible in linear time and logarithmic space (in α), that upon completion only uses three labels; with the corresponding vertex sets being X , Y , and Z .

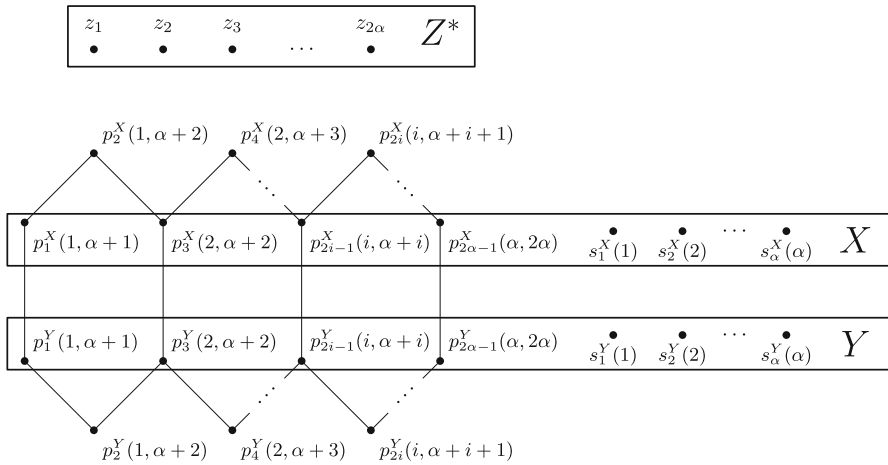


Fig. 2 Illustration of the graph H_α from Lemma 2. All vertices neither in X nor in Y are in Z . The set Z^* induces a clique and edges between Z^* and the remainder of the graph are omitted. For all $u \in V(H_\alpha) \setminus Z^*$, there is an edge between z_j and u if j is not on the list shown after vertex u

2. There is a 2α -coloring of H_α that uses α colors on X and 2α colors on Y ; and a 2α -coloring of H_α that uses 2α colors on X and α colors on Y ; we call such colorings intended.
3. Each coloring of H_α with at most 2α colors is intended.

Proof The graph H_α is constructed as follows. First, we add a clique $Z^* = \{z_1, \dots, z_{2\alpha}\}$ on 2α vertices to H_α . Then, we add two paths on $2\alpha - 1$ vertices, P^X and P^Y , and we denote their vertices in the order in which they appear on the path by $p_1^X, \dots, p_{2\alpha-1}^X$ and by $p_1^Y, \dots, p_{2\alpha-1}^Y$, respectively. We furthermore add two independent sets $S^X = \{s_1^X, \dots, s_\alpha^X\}$ and $S^Y = \{s_1^Y, \dots, s_\alpha^Y\}$ on α vertices each, to H_α . The partition (X, Y, Z) of $V(H_\alpha)$ required by the statement of the lemma will be given by $X = S^X \cup \{p_i^X \mid i \bmod 2 = 1\}$, $Y = S^Y \cup \{p_i^Y \mid i \bmod 2 = 1\}$, and $Z = V(H_\alpha) \setminus (X \cup Y)$.

For all odd $i \in [2\alpha - 1]$, we connect the vertices p_i^X and p_i^Y with an edge. To specify additional adjacencies in H_α , we define a function $f: V(H_\alpha) \setminus Z^* \rightarrow 2^{[2\alpha]}$ as follows, for all $U \in \{X, Y\}$:

$$\text{For all odd } i \in [2\alpha - 1]: f(p_i^U) = \{i', \alpha + i'\}, \text{ where } i' = \lceil i/2 \rceil.$$

$$\text{For all even } i \in [2\alpha - 2]: f(p_i^U) = \{i', \alpha + i' + 1\}, \text{ where } i' = i/2.$$

$$\text{For all } i \in [\alpha]: f(s_i^U) = \{i\}.$$

To finish the construction of H_α , for all $v \in V(H_\alpha) \setminus Z^*$, and all $j \in [2\alpha]$, we add the edge vz_j to H_α if and only if $j \notin f(v)$. We illustrate this construction in Fig. 2.

In each proper 2α -coloring of H_α , we may assume that each vertex z_i , where $i \in [2\alpha]$, received color i . We call such 2α -colorings *canonical*.

The following observation is immediate from the above construction.

Observation 1 *In each proper canonical 2α -coloring of H_α , each $v \in V(H_\alpha) \setminus Z^*$ receives a color in $f(v)$.*

We are now able to show that H_α satisfies Items 2 and 3 of the statement.

Claim 2 *There are precisely two proper canonical 2α -colorings of H_α , one in which colors $[\alpha]$ appear on X and colors $[2\alpha]$ appear on Y , and one in which colors $[2\alpha]$ appear on X and colors $[\alpha]$ appear on Y .*

Proof We exhibit the first type of coloring. Throughout, we make use of Observation 1 which asserts that the values of the function f can be viewed as lists specifying permissible colors for each vertex not in Z^* .

First, we color p_1^X with color 1, which forces color $\alpha + 1$ on its neighbor p_2^X , which forces color 2 on p_3^X . Generally, for odd i , p_i^X is forced to receive color $\lceil i/2 \rceil$, and for even i , p_i^X is forced to receive color $\alpha + i/2$. For each odd i , there is an edge from p_i^X to p_i^Y , which forces color $\alpha + \lceil i/2 \rceil$ on vertex p_i^Y . This in turn forces color $i/2$ on each vertex p_i^Y for even i . Lastly, for all $i \in [\alpha]$, the vertices s_i^X and s_i^Y receive their only permissible color, i .

The second coloring can be obtained symmetrically, starting by assigning vertex p_1^Y color 1. To observe that there are no more than these two proper canonical 2α -colorings of H_α , note that in each such coloring, either p_1^X or p_1^Y has to receive color 1, and once this choice is fixed, the colors of the remaining vertices are forced, as argued in the paragraph above. $\triangleleft$

It remains to give the linear 10-expression constructing H_α which satisfies the properties of Item 1. We show a linear 10-expression constructing a series of graphs H_α^i , for $i \in [\alpha - 1]$, where H_α^i is the subgraph of H_α induced on the vertices

$$\{z_1, \dots, z_i, z_{\alpha+1}, \dots, z_{\alpha+i}, p_1^X, \dots, p_{2i}^X, p_1^Y, \dots, p_{2i}^Y, s_1^X, \dots, s_i^X, s_1^Y, \dots, s_i^Y\}.$$

Our construction works as follows. In each iteration, we introduce the necessary vertices, and immediately add all edges that connect the new vertices to the graph obtained from the previous iteration (if any). At each iteration $i \in [\alpha - 1]$, we introduce the vertices $\{z_i, z_{\alpha+i}\} \cup \bigcup_{U \in \{X, Y\}} \{p_{2i-1}^U, p_{2i}^U, s_i^U\}$. (The last iteration works in the same way, apart from the vertices $p_{2\alpha}^U$ not being introduced.) Before we go into the details, we list the labels and their intended use to create these vertices.

For the vertices in Z^* , there are labels L_{Z^*} , $L_{Z^*}^{+\alpha}$, and $L_{Z^*}^{old}$. The vertices introduced in previous iterations have label $L_{Z^*}^{old}$; while the vertex z_i will receive label L_{Z^*} and the vertex $z_{\alpha+i}$ will receive label $L_{Z^*}^{+\alpha}$ upon creation.

We use the labels L_U , L_U^{prev} , L_U^{old} , where $U \in \{X, Y\}$, as well as the label L_{rest}^{old} to create the vertices in $U \cup V(P^U)$. The label L_U^{old} is meant for vertices in U that have been created in previous iterations, while the label L_{rest}^{old} is meant for vertices in P^U outside of U created in previous iterations. (The lemma requires that upon completion, these two sets of vertices have different labels.) The labels L_U and L_U^{prev} are used in the introduction of the new vertices. The label L_U^{prev} is first used to connect p_{2i-1}^U to its predecessor on the path, and then in the next iteration to connect p_{2i}^U to its successor.

For the sake of clarity, we first show how to construct the initial labeled graph H_α^1 .

1. Introduce vertex z_1 with label $L_{Z^*}^{old}$ and $z_{\alpha+1}$ with label $L_{Z^*}^{+\alpha}$; make $L_{Z^*}^{old}$ adjacent to $L_{Z^*}^{+\alpha}$.
2. For all $U \in \{X, Y\}$, introduce p_1^U with label L_U and p_2^U with label L_U^{prev} ; make L_U adjacent to L_U^{prev} .
3. Make L_X adjacent to L_Y .
4. For all $U \in \{X, Y\}$, relabel L_U to L_U^{old} .
5. For all $U \in \{X, Y\}$, introduce s_1^U with label L_U ; make $L_{Z^*}^{+\alpha}$ adjacent to L_U^{prev} and L_U ; relabel L_U to L_U^{old} .
6. Relabel $L_{Z^*}^{+\alpha}$ to $L_{Z^*}^{old}$.

We now describe how to construct, for $i \in [\alpha - 1]$, the graph H_α^i , assuming that H_α^{i-1} has been constructed. We ensure that after constructing H_α^i , the following invariants hold. (Note in particular that our construction of H_α^1 above achieves this.)

- The vertices in $V(H_\alpha^i) \cap Z^*$ have label $L_{Z^*}^{old}$.
- For all $U \in \{X, Y\}$, the vertices in $V(H_\alpha^i) \cap U$ have label L_U^{old} .
- For all $U \in \{X, Y\}$, the vertex p_{2i}^U has label L_U^{prev} .
- All vertices in $V(H_\alpha^i) \setminus (Z^* \cup X \cup Y)$ have label L_{rest}^{old} .

In the following, we may use $L_{V \setminus Z^*}^{old}$ to denote the collection of labels $L_X^{old}, L_Y^{old}, L_{rest}^{old}$. To construct H_α^i from H_α^{i-1} , do:

1. For all $U \in \{X, Y\}$, introduce p_{2i-1}^U with label L_U ; make L_U adjacent to L_U^{prev} and $L_{Z^*}^{old}$.
2. Make L_X adjacent to L_Y .
3. Introduce z_i with label L_{Z^*} , and $z_{\alpha+i}$ with label $L_{Z^*}^{+\alpha}$.
4. Make L_{Z^*} adjacent to $L_{Z^*}^{+\alpha}, L_{V \setminus Z^*}^{old}, L_{Z^*}^{old}, L_X^{prev}$, and L_Y^{prev} .
5. Make $L_{Z^*}^{+\alpha}$ adjacent to $L_{V \setminus Z^*}^{old}$ and $L_{Z^*}^{old}$.
6. For all $U \in \{X, Y\}$, relabel L_U^{prev} to L_{rest}^{old} .
7. For all $U \in \{X, Y\}$, introduce p_{2i}^U with label L_U^{prev} ; make L_U^{prev} adjacent to $L_U, L_{Z^*}^{old}$, and $L_{Z^*}^{+\alpha}$.
8. For all $U \in \{X, Y\}$, relabel L_U to L_U^{old} .
9. For all $U \in \{X, Y\}$, introduce s_i^U with label L_U ; make L_U adjacent to $L_{Z^*}^{old}$ and $L_{Z^*}^{+\alpha}$; relabel L_U to L_U^{old} .
10. Relabel L_{Z^*} and $L_{Z^*}^{+\alpha}$ to $L_{Z^*}^{old}$.

Finally, to obtain H_α from $H_\alpha^{\alpha-1}$, we perform all above steps with $i = \alpha$ that do not involve the (non-existing) vertices $p_{2\alpha}^U$, i.e., all steps except the one described in Item 7. To obtain the desired labeling, we relabel $L_{Z^*}^{old}$ to L_{rest}^{old} . Indeed, the set X now has label L_X^{old} , the set Y the label L_Y^{old} , and the set Z the label L_{rest}^{old} . $\square$

We build $H_{\alpha,B}$ from H_α by adding a clique with $B - 2\alpha$ vertices, and making these vertices adjacent to all vertices in H_α .

Theorem 3 COLORING parameterized by linear clique-width is XNLP-complete.

Proof Membership in XNLP was shown in Lemma 1. We prove hardness via a reduction from MINIMUM MAXIMUM OUTDEGREE with pathwidth as parameter. This problem was shown to be XNLP-complete in [1].

Suppose, we are given a graph $G = (V, E)$ with for each edge a weight $w(e)$, given in unary, with target value r . Denote for each $v \in V$, the total weight of edges incident to v by $d(v) = \sum_{\{v,w\} \in E} w(\{v, w\})$. Denote the total weight of all edges by $T = \sum_{e \in E} w(e)$. Build a graph G' as follows. Replace each edge e by the gadget $H_{w(e), 2T}$. For an edge $\{v, w\}$, write $V_{v,e}$ for the set X of the gadget and $V_{w,e}$ for the set Y of the gadget.

To the disjoint union of the gadgets, we add the following edges and vertices. For each vertex v , add edges between vertices in $V_{v,e}$ and $V_{v,e'}$ for all $e \neq e'$, and take a clique C_v with $2T - d(v) - r$ vertices, and add edges from all vertices in C_v to all vertices in $V_{v,e}$. Write V_v for the union of all sets $V_{v,e}$ and C_v .

Claim 3 G has an orientation with maximum weighted outdegree r , if and only if G' can be colored with $2T$ colors.

Proof Suppose we have an orientation of G with maximum weighted outdegree r . To each edge e , we associate a set Γ_e of $2w(e)$ colors, such that these sets are disjoint; we can do this as the total number of colors is sufficiently large. Now, for each edge $e = \{v, w\} \in E$, we use the colors of Γ_e to color the vertices in the gadget of e . If the edge e is directed from v to w , then we color the gadget in such a way that $2w(e)$ colors are used for $V_{v,e}$, and $w(e)$ colors are used from $V_{w,e}$; and if the edge is directed from w to v , then we color the gadget in such a way that $w(e)$ colors are used for $V_{v,e}$, and $2w(e)$ colors are used from $V_{w,e}$.

For each vertex v , the total number of colors used for vertices in sets $V_{v,e}$ over all edges e incident to v equals $d(v)$ plus the total weight of edges directed out of v ; by assumption, the latter term is at most r . We now can color each vertex in C_v by a color not used in the sets $V_{v,e}$, as we have $2T - d(v) - r$ colors left.

Suppose we have a coloring of G' with $2T$ colors. Consider an edge $e = \{v, w\}$, and the corresponding gadget, which is a copy of $H_{w(e), 2T}$. Due to the clique on $2T - 2w(e)$ vertices which see all vertices on the copy of $H_{w(e)}$ in $H_{w(e), 2T}$, its $H_{w(e)}$ -copy has to be colored with at most 2α colors. The gadget property described in Lemma 2 tells us that we either use $w(e)$ colors for vertices in $V_{v,e}$ and $2w(e)$ colors for vertices in $V_{w,e}$, or $2w(e)$ colors for vertices in $V_{v,e}$ and $w(e)$ colors for vertices in $V_{w,e}$. In the former case, direct e from w to v , and in the latter case, direct e from v to w .

We claim that this orientation has an outdegree that is at most r . Consider a vertex v . Note that each of the sets C_v , and $V_{v,e}$ for all edges e incident to v uses a different set of colors. The number of colors used over all sets $V_{v,e}$ is at least $d(v)$ plus the total weight of all edges directed out of v . For the clique C_v , we use $2T - d(v) - r$ colors. Thus, the total weight of all edges directed out of v is at most $2T - (2T - d(v) - r) - d(v) = r$. $\triangleleft$

Claim 4 G' has linear clique-width at most $k + O(1)$, where k is the pathwidth of G . We can construct the corresponding expression using $f(k) + O(\log(n))$ space.

Proof Suppose we have a path decomposition of G of width k . Transform it to a nice path decomposition, with introduce vertex, forget, and introduce edge nodes.

We use the following labels: 10 labels for building gadgets, a label γ_j for $j \in [1, k + 1]$ a label δ for vertices that will not receive new neighbors.

We visit the bags of the path decomposition in order. For each vertex in the current bag, we have a unique number in $[1, k + 1]$. We describe how to build the expression for G' .

Suppose we have an introduce vertex node X_i , that introduces vertex v . Let j be the smallest integer in $[1, k + 1]$ not assigned to a vertex in X_{i-1} , with $j = 1$ when $i = 1$. Assign j to v . Build the clique C_v , giving this clique label γ_j .

Suppose we have a forget node, that forgets vertex v . Suppose j is assigned to v . Now, recolor γ_j to δ .

Suppose we have an introduce edge node, that introduces the edge $\{v, w\}$ with weight α . Build the gadget $H_{\alpha, B}$ with the gadget labels. Now, say that the vertices in X have color ϵ_1 , and the vertices in Y have label ϵ_2 . Relabel all other gadget label to δ (these vertices do not get additional neighbors). The vertices in X , with gadget label ϵ_1 will form $V_{v, \epsilon}$, and the vertices in Y , with gadget label ϵ_2 will form $V_{w, \epsilon}$. Suppose the index of v is $j_1 \in [1, k + 1]$, and the index of w is $j_2 \in [1, k + 1]$. Add the edges between label class ϵ_1 and label class γ_{j_1} . Add the edges between label class ϵ_2 and label class γ_{j_2} . Now, relabel ϵ_1 to γ_{j_1} , and relabel ϵ_2 to γ_{j_2} .

One can verify that the resulting graph is G' . We used $k + O(1)$ labels, so the linear clique-width of G' is $k + O(1)$. $\triangleleft$

This finishes the proof of the theorem. $\square$

Finally, we consider the MAXIMUM REGULAR INDUCED SUBGRAPH problem and show that this problem is XNLP-complete with linear clique-width as parameter.

Theorem 4 MAXIMUM REGULAR INDUCED SUBGRAPH *parameterized by linear clique-width is XNLP-complete.*

Proof We first show membership in XNLP. We keep in memory a certificate for each label class corresponding to the number of vertices in the label class that are chosen in the induced subgraph, along with their current degree. If the current degree of vertices in the same label class is not uniform then we can already reject, and we therefore only need to store a single degree per label class. The choice of a vertex for our induced subgraph is nondeterministic. We accept if at the end of the linear clique-width expression, the degree of all non empty label classes is the target degree D and the total amount of chosen vertices is at least W .

We prove hardness via a reduction from CHAINED POSITIVE CNF- SAT. Given an instance $\phi, p, X_1, \dots, X_r, k, (X_{i,j})_{i \in [r], j \in [k]}$ of CHAINED POSITIVE CNF- SAT, let $C_1, \dots, C_m$ be the clauses in the Boolean formula ϕ and let $C[i]$ be the number of clauses that use variables of X_i . We set $D = \max \{2(1 + \max_{i,j} |X_{i,j}|), 2\lceil C[i]/2 \rceil\}$ and choose N to be a large enough integer, e.g. $N = 6krD^3 + AD^3$ where A is the total number of literals over all clauses of ϕ .

We need the following gadgets to create a graph G .

Degree filling gadgets The following gadget will be used to increase the degree of the variable choice gadgets. Suppose a pair of vertices v, v' has been given. We add five vertices w_0, w_1, w_2, w_3, w_4 , a $(D - 2)$ -clique and a $(D - 1)$ -clique. All vertices in the $(D - 2)$ -clique are adjacent to w_0, w_1 and w_2 . All vertices in the $(D - 1)$ -clique are adjacent to w_3 and w_4 . Let w_0 be adjacent to v, v' , w_1 be adjacent to both w_2 and w_3 , and w_2 be adjacent to w_4 . See Fig. 3.

Fig. 3 Degree filling gadget

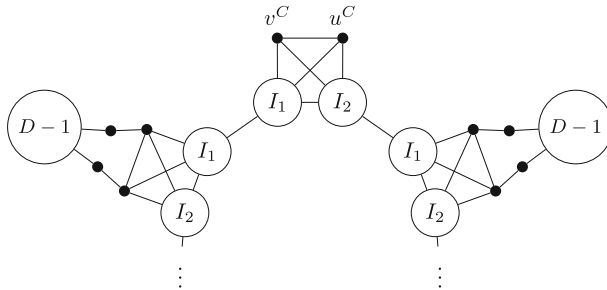
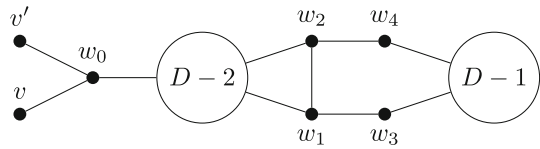


Fig. 4 Clause gadget

Variable choice gadget For each $X_{i,j}$, we add $|X_{i,j}| - 1$ pairs of vertices v, v' . We add D degree filling gadgets to each pair of vertices, resulting in a gadget we denote by $\widehat{X_{i,j}}$.

Clause gadget For each clause C , we add $2N$ vertices $z_1, \dots, z_{2N}$ and $2N$ independent sets $I_1, \dots, I_{2N}$ each of size $\frac{D}{2} - 1$. For $i \in [N]$, we make z_{2i-1} and z_{2i} adjacent to each other, and adjacent to I_{2i-1} and I_{2i} . The independent sets are placed in a cycle: for $i \in [1, 2N - 1]$, I_i is adjacent to I_{i+1} , and I_{2N} is adjacent to I_1 .

We denote z_1, z_2 by v^C, u^C . For each $i \in [2, N]$, we add two vertices y, y' and a $(D - 1)$ -clique, make y and y' adjacent to the clique, and add edges $yz_{2i-1}, y'z_{2i}$. See Fig. 4. Note that all vertices have degree D except for v^C and u^C which have degree $D - 1$.

Degree constraint gadget For a single vertex v , to allow increasing its degree by 2, we add two vertices a, b adjacent to v , to each other and to a $(D - 1)$ -clique.

To allow increasing the degree of v by 2ℓ with $\ell > 1$, we add 2ℓ vertices $a_1, \dots, a_{2\ell}$ and 2ℓ independent sets $I_1, \dots, I_{2\ell}$ each of size $\frac{D}{2} - 1$. For $i \in [\ell]$, a_{2i-1} and a_{2i} are adjacent to each other and are both adjacent to v, I_{2i-1} and I_{2i} . The independent sets are placed in a cycle: I_1 is adjacent to $I_{2\ell}$ and for $i \in [1, 2\ell - 1]$, I_i is adjacent to I_{i+1} .

Variable reading gadget For each literal $a \in X_{i,j}$ that appears in a clause C , we add a vertex v_a^C . This is adjacent to the vertices v^C and u^C from the clause gadget and to all pairs of vertices v, v' of the variable gadget $\widehat{X_{i,j}}$. If literal a corresponds to the variable of index $p \in [0, |X_{i,j}| - 1]$ in $X_{i,j}$, then we add a degree constraint gadget to allow increasing the degree of v_a^C by $D - 2(p + 1)$ which is even and at least 2 by definition of D .

Together, the gadgets above constitute graph G . We set $W' = (N(2D + 1) - D - 1)$ (the size of a clause gadget) and set the required size of the D -regular induced subgraph that we are looking for to $W = mW'$. We have chosen N sufficiently large such that all clause gadgets need to have at least one vertex included in the subgraph (as N is larger than the size of all non-clause gadgets combined).

Claim 5 $\text{lcw}(G) \leq 2k + \mathcal{O}(1)$

Proof We have an *inactive* label for vertices that are already adjacent to all of their neighbourhood. All gadgets can be constructed using a constant amount of fresh labels, and we never construct two gadgets of the same type simultaneously. Each clause only depends on variables from X_i and X_{i+1} for some i , from the structure of the considered formula ϕ . We construct gadgets for increasing i . There are $2k$ variable choice gadgets $\widehat{X_{i,j}}$ and $\widehat{X_{i+1,j}}$ (with $j \in [k]$) and we reserve a separate label for each such gadget. We create the variable gadgets, and put all vertices from the degree filling gadgets to inactive (so only the pairs of vertices v, v' keep the label of the gadget). This way, we can keep attaching variable reading gadgets to them, clause by clause. Once all clauses containing a literal of $X_{i,j}$ have had their gadget constructed, we can relabel the vertices of $\widehat{X_{i,j}}$ to the inactive label. $\triangleleft$

Claim 6 *If the SAT instance is satisfiable, then there is a D -regular induced subgraph of G of size at least W .*

Proof If the variable of index p in $X_{i,j}$ is set to true in the SAT instance, we pick p pairs of vertices v, v' in $\widehat{X_{i,j}}$ into the induced subgraph. We pick all clause gadgets. For each clause, since it is satisfied, it has a satisfied literal a . We pick the vertex v_a^C and its degree constraint gadget. Now all picked vertices have degree D except for the vertices of the variable choice gadgets. By definition of D , the current degree of the picked vertices in the variable choice gadgets must be less than D , so we can complete their degree using the degree filling gadgets. From picking all clause gadgets, our induced subgraph has size at least W . $\square$

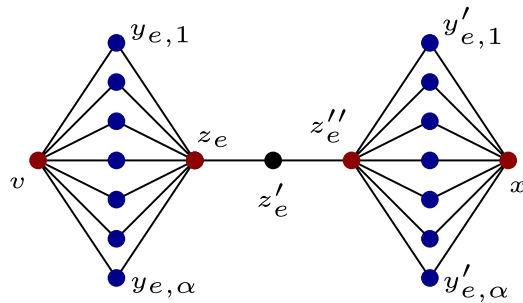
Claim 7 *If there is a D -regular induced subgraph of G of size at least W , then the SAT instance is satisfiable.*

Proof Let H denote the D -regular induced subgraph of G of size at least W . Since H is of size at least W it must include a vertex of each clause gadget. By design of the clause gadgets, all of their vertices must be included in H for it to be D -regular. Furthermore, for each clause C , the vertices v^C, u^C must be adjacent to a vertex outside the clause gadget. By construction, this vertex can only be a vertex of type v_a^C . Since this vertex has degree D in H , it must be adjacent to some vertex of the corresponding degree constraint gadget. By design of the degree constraint gadget, all of its vertices must be included in H for it to be D -regular. Now since v_a^C has degree D , it must be adjacent to exactly $2p$ vertices of the variable choice gadget. This means that the variable of index p satisfies the clause C , but also that H includes exactly $2p$ vertices of $\widehat{X_{i,j}}$ (not counting the degree filling gadgets).

Now for each $X_{i,j}$, denoting by r the number of vertices v, v' of $\widehat{X_{i,j}}$ that are in H , we set the variable of index $\lfloor \frac{r}{2} \rfloor$ to true. $\triangleleft$

The two claims above show that we have constructed an equivalent instance of MAXIMUM REGULAR INDUCED SUBGRAPH. It is not hard to see that the transformation can be carried out in $f(k) \log n$ space, and thus the theorem follows. $\square$

Fig. 5 Edge gadget for the edge $\{v, x\}$ from the proof of Theorem 5



4.2 Variants of Dominating Set Parameterized by Pathwidth

In this section we prove the following theorem.

Theorem 5 CAPACITATED RED-BLUE DOMINATING SET and CAPACITATED DOMINATING SET parameterized by pathwidth are XNLP-complete.

Proof We first show membership in XNLP for CAPACITATED RED-BLUE DOMINATING SET. For each red vertex, we guess if it is in the dominating set, and for each edge from a chosen red vertex to a blue neighbor, we guess if it is used for dominating. We do this while going through the path decomposition from left to right. We need to keep track of which blue vertices are already dominated, which red vertices are in the dominating set plus their remaining capacity, and the total number of vertices in the dominating set so far. We may assume that the remaining capacities are never larger than the number of blue vertices; therefore, we only need to store $O(\log n)$ bits per vertex in the current bag. Membership in XNLP follows similarly for CAPACITATED DOMINATING SET.

Hardness follows by a reduction from CIRCULATING ORIENTATION (defined in Sect. 4.1). Suppose that we are given an input of CIRCULATING ORIENTATION, say a graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{N}$. We assume that these weights are given in unary. Note that in a solution, the total weight of edges directed towards a vertex v and the total weight of the edges directed out of v should both equal $\sum_{\{v,x\} \in E} w(\{v, x\})/2$.

We build a graph as follows. For each vertex $v \in V$, we create a vertex v , colored red, in H . We give v a private blue neighbor v' . The capacity of v equals $1 + \sum_{\{v,x\} \in E} w(\{v, x\})/2$. We can assume this capacity is integral, otherwise there is no solution to the instance (G, w) . Each edge $e = \{v, x\} \in E$ is replaced by the following gadget. Suppose $w(\{v, x\}) = \alpha \in \mathbb{N}$. We create $2\alpha + 3$ vertices, called $y_{e,1}, y_{e,2}, \dots, y_{e,\alpha}, z_e, z'_e, z''_e, y'_{e,1}, \dots, y'_{e,\alpha}$. The edge e is replaced by the subgraph shown in Fig. 5. The vertices z_e and z''_e are red, and all other new vertices are blue. We give the new red vertices z_e and z''_e a capacity that equals their degree.

Let $H = (V_H, E_H)$ be the resulting red-blue colored graph, with $c(v)$ the capacity of a red vertex $v \in V_H$. We claim that H has a dominating set of size $|V| + |E|$ for which each chosen red vertex dominates at most its capacity many blue vertices, if and only if G has a circulating orientation.

Suppose first that we have a set S of red vertices with $|S| \leq |V| + |E|$, and an assignment of blue vertices to neighbors in S , such that no red vertex has more than its capacity number of vertices assigned to it.

Each vertex that is a copy of a vertex from V must belong to S , as they have a private blue neighbor. For each edge e , either z_e or z_e'' must be in S , to dominate z_e' . This gives in total already $|V| + |E|$ vertices, so no edge can have both z_e and z_e'' in S . For each edge $e = \{v, x\}$, if $z_e \in S$, then orient the edge from v to x in G ; if $z_e'' \in S$, then orient the edge from x to v . Now, for each $v \in V$, the total weight of incoming edges of the orientation can be at most $c(v) - 1$, since v must also dominate its private neighbor. By definition, $c(v) - 1 = \sum_{\{v,x\} \in E} w(\{v, x\})/2$. This means that for each vertex, the total weight of incoming edges is at most half the total weight of incident edges; it follows that this total weight must be equal, because when there is a vertex for which this weight is smaller, then there must be another vertex for which it is larger. So, we have an orientation that is a circulation.

Suppose now that we have a circulation that is an orientation. Add each original vertex $v \in V$ to S , and for each edge $e = \{v, x\}$, place z_e in S when the edge is oriented from v to x and otherwise place z_e'' in S . Red vertices on edge gadgets dominate all their neighbors; red original vertices dominate their private neighbor and all not yet dominated blue neighbors. This gives a dominating set where each red vertex in S dominates precisely its capacity many neighbors, as desired.

Finally, we show that we can build a log-space transducer that transforms a path decomposition of G of width ℓ to one of H with width at most $\ell + 2$. We first ensure that the path decomposition of G is nice (which can be done via a log-space transducer). We pass through the bags from left to right. For a forget bag in the path decomposition of G , we take the same bag for H . For an introduce bag $X_i = X_{i-1} \cup \{v\}$, we loop through the vertices in X_{i-1} one-by-one, say these are $x_1, \dots, x_r$. For each $j \in [r]$, if $\{v, x_j\} \in E$, then we add the following bags (in order):

$$X_i \cup \{z_e, y_{e,1}\}, X_i \cup \{z_e, y_{e,2}\}, \dots, X_i \cup \{z_e, y_{e,w(e)}\}, X_i \cup \{z_e, z_e'\}, X_i \cup \{z_e', z_e''\}, \\ X_i \cup \{z_e'', y_{e,1}'\}, X_i \cup \{z_e'', y_{e,2}'\}, \dots, X_i \cup \{z_e'', y_{e,w(e)}'\}.$$

One can verify that this gives a path decomposition of H . The width has increased by at most 2.

A standard transformation now shows that CAPACITATED DOMINATING SET is also XNLP-hard with pathwidth as parameter. Given an instance (G, w) of CAPACITATED RED- BLUE DOMINATING SET, we build an equivalent instance of CAPACITATED DOMINATING SET. We give each blue vertex capacity zero. We add two new vertices x and x' , with x' of degree one and x adjacent to all red vertices and to x' . The capacity of x is equal to the number of red vertices plus 2. We increase the target size of the solution by one (and remove all colors). The pathwidth has gone up by at most one.

□

A similar reduction can be used to show XNLP-hardness of CAPACITATED VERTEX COVER.

Theorem 11 CAPACITATED VERTEX COVER *parameterized by pathwidth is XNLP-complete.*

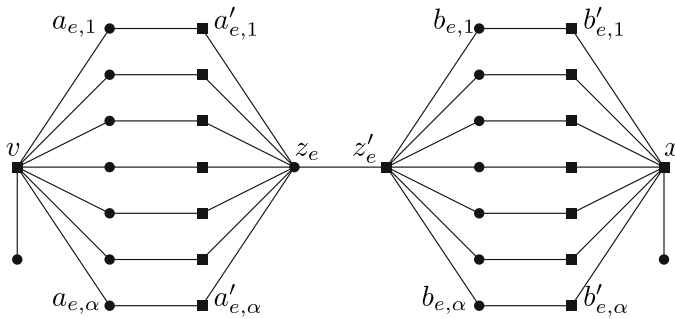


Fig. 6 Edge gadget for the edge $\{v, x\}$ from the proof of Theorem 11. Placing the vertices represented by a square in the vertex cover corresponds to directing edge e from v to x

Proof Membership follows in a similar way as for CAPACITATED DOMINATING SET: guess for each vertex if it is in the vertex cover, and for each edge which of its endpoints covers it. We keep track of the vertices in the vertex cover plus their capacity, and the total number of vertices in the vertex cover so far; this is again $O(\log n)$ bits per vertex in the current bag.

Hardness follows by a reduction from CIRCULATING ORIENTATION. Suppose we are given a graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{N}$. Let H be obtained by the following steps. First, we give every vertex in V a new pendant vertex (neighbour of degree one.) For each edge $e = \{v, x\} \in E$, with $w(e) = \alpha$, we create $4\alpha + 2$ new vertices: for each $i \in [1, \alpha]$, we have vertices $a_{e,i}$, $a'_{e,i}$, $b_{e,i}$ and $b'_{e,i}$, and we have vertices z_e and z'_e . For each $i \in [1, \alpha]$, we have edges $\{v, a_{e,i}\}$, $\{a_{e,i}, a'_{e,i}\}$, $\{a'_{e,i}, z_e\}$, $\{z'_e, b_{e,i}\}$, $\{b_{e,i}, b'_{e,i}\}$, and $\{b'_{e,i}, x\}$, and we have an edge $\{z_e, z'_e\}$. An example for the gadget for an edge e with weight $w(e) = 7$ is given in Fig. 6.

We set the capacities as follows. Each vertex in $v \in V$ has capacity $1 + \sum_{\{v,x\} \in E} w(\{v, x\})/2$. The pendant vertices have capacity 0. All other (new) vertices have a capacity equal to their degree, i.e., vertices z_e and z'_e have capacity $w(e) + 1$, and all vertices of the form $a_{e,i}$, $a'_{e,i}$, $b_{e,i}$ and $b'_{e,i}$ have capacity 2.

Let $H = (V_H, E_H)$ be the resulting graph with capacity function c . As H and c can be constructed in logarithmic space, and one easily observes that $\text{pw}(H) = \text{pw}(G) + O(1)$, the XNLP-hardness result follows from the following claim.

Claim 8 G with edge weights given by w has a circulating orientation, if and only if H with vertex capacities given by c has a capacitated vertex cover of size at most $|V| + \sum_{e \in E} (2w(e) + 1)$.

Proof Suppose we have a circulating orientation of G . Build a capacitated vertex cover S for H as follows. Place all original vertices $v \in V$ in S . If an edge $e = \{v, x\}$ is oriented from v to x , then add to S all vertices of the form $a'_{e,i}$, z'_e and $b'_{e,i}$. This is illustrated in Fig. 6: the vertices placed in S are represented by a square. If $e = \{v, x\}$ is oriented from x to v , then add to S all vertices of the form $a_{e,i}$, z_e and $b_{e,i}$. All vertices in S in the edge gadgets cover all incident edges. For each $v \in V$, the number of incident edges not covered yet equals one (for the pendant vertex) plus the outdegree of v in the circulation, which equals its capacity.

Suppose we have a capacitated vertex cover S of H of size at most $|V| + \sum_{e \in E} (2w(e) + 1)$. First, note that all vertices $v \in V$ are in S , as the edge from v to its pendant edge must be covered. Each edge gadget for an edge e has a matching with $2w(e) + 1$ edges, so we use at least $2w(e) + 1$ vertices in the gadget of e in the vertex cover. Due to the size bound on the vertex cover, we also cannot use more than $2w(e) + 1$ vertices per edge gadget. This implies that exactly one vertex per edge of such a matching must be chosen.

Now, for each edge gadget for edge $e = \{v, x\}$, we have two choices for which vertices from the gadget are in S : either we have all vertices of the form $a'_{e,i}$, z'_e and $b'_{e,i}$ in S , or we have all vertices of the form $a_{e,i}$, z_e and $b_{e,i}$ in S — any other choice of $2w(e) + 1$ vertices in the gadget would leave at least one edge between two vertices inside the gadget (i.e., vertices in $V_H \setminus V$) uncovered. In the former case, direct e from v to x ; in the latter case, direct e from x to v . The vertices in V cover their pendant edge, and the incident edges from the gadgets that are not covered by the other endpoint: these are precisely the incident edges from gadgets of edges that are directed out of v . This implies that in the orientation, each vertex has weighted outdegree at most $\sum_{\{v,x\} \in E} w(\{v, x\})/2$. We can use the same argument as in the proof of Theorem 5, and conclude that we have a circulating orientation of G . $\triangleleft$

This concludes the proof of Theorem 11. $\square$

4.3 Problems Parameterized by Logarithmic Pathwidth or Logarithmic Linear Clique-Width

In this section, we prove the result restated below.

Theorem 6 *When parameterized by logarithmic pathwidth or logarithmic linear clique-width, INDEPENDENT SET, DOMINATING SET, q -LIST-COLORING for $q > 2$, and ODD CYCLE TRANSVERSAL are XNLP-complete, and FEEDBACK VERTEX SET is XNLP-hard.*

The problem definitions for the parameter logarithmic pathwidth are given below; those for the parameter logarithmic clique-width can be obtained by replacing ‘path decomposition’ by ‘linear clique-width expression’.

INDEPENDENT SET (IS) / log pathwidth

Input: A graph $G = (V, E)$, a path decomposition of G of width ℓ , and an integer k .

Parameter: $\lceil \ell / \log |V| \rceil$

Question: Is there a subset S of V such that $E \cap S \times S = \emptyset$ and $|S| \geq k$?

DOMINATING SET (DS) / log pathwidth

Input: A graph $G = (V, E)$, a path decomposition of G of width ℓ , and an integer k .

Parameter: $\lceil \ell / \log |V| \rceil$

Question: Is there a subset S of V such that $N[S] = V$ and $|S| \leq k$?

q -LIST- COLORING / log pathwidth

Input: A graph $G = (V, E)$, a path decomposition of G of width ℓ , and lists of available colors among $[q]$ for each vertex of V .

Parameter: $\lceil \ell / \log |V| \rceil$

Question: Is there a proper coloring of G which assigns to each vertex of G a color of its list ?

ODD CYCLE TRANSVERSAL (OCT) / log pathwidth

Input: A graph $G = (V, E)$, a path decomposition of G of width ℓ , and an integer k .

Parameter: $\lceil \ell / \log |V| \rceil$

Question: Is there a subset S of V such that $G - S$ is bipartite and $|S| \leq k$?

Since $\text{lcw} \leq \text{pw} + 2$, a reduction showing hardness for the parameter (logarithmic) pathwidth also shows hardness for the parameter (logarithmic) linear clique-width. Conversely, showing membership for the parameter (logarithmic) linear clique-width shows membership for the parameter (logarithmic) pathwidth. Since $\text{IS}/\log \text{pw}$ and $\text{DS}/\log \text{pw}$ are already known to be XNLP-complete, we can already conclude that $\text{IS}/\log \text{lcw}$ and $\text{DS}/\log \text{lcw}$ are XNLP-hard. All membership claims in Theorem 6 follow from the following lemma.

Lemma 3 *When parameterized by logarithmic linear clique-width, INDEPENDENT SET, DOMINATING SET, q -LIST- COLORING for $q > 2$, and ODD CYCLE TRANSVERSAL are in XNLP.*

Proof The algorithms will keep in memory a certificate of constant size for each of the label classes of the linear clique-width expression. The certificates are as follows.

IS For each label class, we record whether or not it contains a vertex in the independent set.

DS For each label class, we record whether or not it contains a vertex of the dominating set, and whether or not it contains a vertex that is not dominated.

q -LIST- COLORING For each label class and for each color, we record whether or not the label class contains a vertex with this color.

OCT In addition to constructing an odd cycle transversal, we construct a bipartition of the remaining graph. For each label class, for each side of the bipartition, we record whether or not there is a vertex in both the label class and the side of the bipartition.

If there are at most k labels, then the above description uses $O(k \log n)$ space. Membership now follows from applying dynamic programming using the above fingerprints. $\square$

It remains to prove the hardness results, which we do in the next two lemmas.

Lemma 4 *q -LIST- COLORING and q -COLORING parameterized by logarithmic pathwidth are XNLP-hard.*

Proof We reduce from CHAINED POSITIVE CNF- SAT to q - LIST- COLORING parameterized by logarithmic pathwidth.

Given an instance $\phi, p, X_1, \dots, X_r, k, (X_{i,j})_{i \in [r], j \in [k]}$ of CHAINED POSITIVE CNF- SAT, suppose that $C_1, \dots, C_m$ are the clauses in ϕ .

For each $X_{i,j}$ we add dummy variables until $|X_{i,j}| = q^{t_j}$ for some integer t_j , increasing its size by at most a multiplicative factor of q . We enforce that the dummy variables will not appear in any solution by adding a clause containing all initial variables for each $X_{i,j}$.

Variable choice gadget For each $X_{i,j}$, we add vertices $x_{i,j}^1, \dots, x_{i,j}^{t_j}$ with lists $[q]$. Together they encode the index of a variable in $X_{i,j}$ in base q . We denote this gadget by $\widehat{X_{i,j}}$.

Clause gadget For a clause C , with literals $a_1, \dots, a_\ell$, we have a path of length $\ell + 2$, with vertices $p_0, p_1, \dots, p_\ell, p_{\ell+1}$. The vertex p_0 is forced to have color 2, $p_{\ell+1}$ is forced to have color 2 if ℓ is even, and color 3 if ℓ is odd. Vertices $p_1, \dots, p_\ell$ have list $[3]$. Note that by construction at least one of them will have to be colored 1.

Variable reading gadget For a literal a with associated vertex p , such that $a \in X_{i,j}$, we read the chosen variable of $X_{i,j}$ using a *connector* which will allow the color of p to be 1 only if the coloring encoding a appears on $(x_{i,j}^1, \dots, x_{i,j}^{t_j})$. This is done with *color obstruction gadgets*. A *color obstruction gadget* for color $\alpha \in [q]$ on vertex x can take two forms:

- If $\alpha = 1$, we have a vertex w_y , adjacent to x , for each $y \in [q] \setminus \{1\}$, with list $\{1, y\}$. We add a vertex w adjacent to the w_y and to p , with list $[q]$.
- If $\alpha \neq 1$, we have vertices w_y and w'_y for each $y \in [q] \setminus \{1\}$. We make w_y adjacent to x and w'_y . We give to w_y list $\{\alpha, 1\}$, and to w'_y list $\{y, 1\}$. Finally, we add a vertex w adjacent to vertices p and w'_y , with full list.

The connector of p consists of color obstructions gadgets on each vertex of $\widehat{X_{i,j}}$, for all colors but the one picked for the encoding of a . (In the following, we use the following notion: For a graph G with vertex lists, two disjoint vertex sets X and Y , we say that a proper list coloring γ of $G[X]$ can be *extended to Y* if there is a proper list coloring of $G[X \cup Y]$ whose restriction to X equals γ .)

Claim 9 Consider the connector to a vertex p encoding a variable z of $X_{i,j}$ and a coloring of $\widehat{X_{i,j}}$.

1. Any coloring on $\widehat{X_{i,j}}$ and any color $c \in \{2, 3\}$ on p can be extended to the connector.
2. The coloring encoding z on $\widehat{X_{i,j}}$ and color 1 on p can be extended to the connector.
3. In any coloring of the connector, if p is colored with 1, then the coloring encoding z appears on $\widehat{X_{i,j}}$.

See [23, Lemma 11] for proof.

The graph G consists of variable choice gadgets for each $X_{i,j}$, clause gadgets for each clause of $\bigwedge_i \phi(X_i, X_{i+1})$, with each literal vertex having its connector.

Claim 10 If our CHAINED POSITIVE CNF- SAT instance is satisfiable, then G has a proper list-coloring.

Proof We know how to choose exactly one variable $s_{i,j}$ per $X_{i,j}$ to satisfy the given formula. For each $\widehat{X_{i,j}}$, we color it with the encoding of $s_{i,j}$. For each clause, we pick an arbitrary satisfied literal and give the vertex representing it color 1. Its connector can be colored by Claim 9 because the encoding on the variable choice gadget must correspond to the satisfied literal.

The rest of the clause gadget can then be colored greedily. By Claim 9, the coloring can be extended to connectors of vertices not colored with 1. $\triangleleft$

Claim 11 *If G has a proper list-coloring, the CHAINED POSITIVE CNF- SAT instance is satisfiable.*

Proof We choose set the variables $s_{i,j}$ encoded by the coloring on $\widehat{X_{i,j}}$ to true. For each clause gadget, by construction, there must be a vertex with color 1; denote by z its associated variable. By Claim 9, the coloring of its connected variable choice gadget $\widehat{X_{i,j}}$ must be the encoding of z . Hence the clause is satisfied, since the encoded variable is the one we set to true. $\triangleleft$

Let $t = \max_j t_j$. We have $t \leq \log_q(n) + 1 = \mathcal{O}(\log(n))$, where $n = |V|$.

Claim 12 $\text{pw}(G) \leq 2kt + 3$

Proof For each $i \in [r - 1]$, the bags corresponding to $\phi(X_i, X_{i+1})$ contain the vertices of $\widehat{X_{i,j}}$ and $\widehat{X_{i+1,j}}$ for $j \in [k]$ (at most $2kt$). Then each clause gadget together with its connectors can be swept with 4 vertices at a time. $\triangleleft$

To reduce from q -LIST-COLORING to q -COLORING, we just add a q -clique, associate each of its vertices to a color, and make them adjacent to all vertices that did not contain this color in their list. This increases the pathwidth by at most q . $\square$

Lemma 5 ODD CYCLE TRANSVERSAL and FEEDBACK VERTEX SET parameterized by logarithmic pathwidth are XNLP-hard.

Proof We reduce from CHAINED POSITIVE CNF- SAT to ODD CYCLE TRANSVERSAL parameterized by logarithmic pathwidth.

Given an instance $\phi, p, X_1, \dots, X_r, k, (X_{i,j})_{i \in [r], j \in [k]}$ of CHAINED POSITIVE CNF- SAT, we denote $\phi = \bigwedge_{i \in [m]} C_i$.

For each $X_{i,j}$, we add dummy variables until $|X_{i,j}| = 3^{t_j}$ for some integer t_j , increasing its size by at most 3. They are discarded by adding a clause containing all initial variables for each $X_{i,j}$.

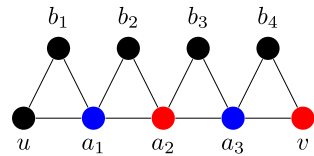
Variable choice gadget For each $X_{i,j}$, we add triangles $T_{i,j}^1, \dots, T_{i,j}^{t_j}$, where each triangle has vertices v_0, v_1, v_2 . The deleted vertices of the triangles will encode the index of a variable in $X_{i,j}$ in base 3. We denote this gadget by $\widehat{X_{i,j}}$.

Clause gadget For a clause C , with literals $a_1, \dots, a_\ell$, we have a cycle of odd length $\ell' = 2 \lfloor \frac{\ell}{2} \rfloor + 1 \in \{\ell, \ell + 1\}$ with vertices $c_1, \dots, c_{\ell'}$.

Arrow gadget An arrow from u to v (see Fig. 7) consists of additional vertices $a_1, a_2, a_3, a_4, b_1, b_2, b_3$ and edges $ua_1, ub_1, a_1b_1, b_1a_2, b_1b_2, a_2b_2, b_2a_3, b_2b_3$,

Fig. 7 The arrow $A(u, v)$.

Vertices in blue form the passive OCT of $A(u, v)$, and those in red form the active OCT of $A(u, v)$ u (color figure online)



$a_3b_3, b_3a_4, b_3v, a_4v$. Note that if u is not in an optimal odd cycle transversal, then b_1 and b_3 must be. The arrow is then said to be “passive”. However, if u is in the odd cycle transversal, then we can also have b_2 and v . The arrow is then said to be “active”. Intuitively, we got v for free from having u .

Negation gadget A negation from u to v consists of additional vertices a_1, a_2 and edges $uv, ua_1, ua_2, va_1, va_2$. Note that we only need to pick one of u and v in our odd cycle transversal to hit the odd cycles of this gadget, and we have to pick at least one.

Variable reading gadget Consider a variable $x \in X_{i,j}$ with index $b_1 \dots b_{t_j}$, appearing in some clause C as literal a_z . We add vertices w_1 and w_2 , make them adjacent to each other and, for each $a \in [t_j]$, to a vertex u_a which is the endpoint of an arrow from vertex v_{b_a} in triangle $T_{i,j}^a$. Finally, we have a negation gadget from w_1 to the vertex c_z in the clause gadget of C .

These gadgets constitute a graph G representing ϕ . Let α be the number of arrows in our construction, and ν be the number of negations. We set $W = 2\alpha + \nu + r \sum_{j \in [k]} t_j$.

Claim 13 *If the SAT instance can be satisfied then there is an odd cycle transversal of size W .*

Proof For the triangles in the variable choice gadgets, we pick the vertices corresponding to the encoding of the variable that is assigned. This amounts to $r \sum_{j \in [k]} t_j$ vertices.

For each arrow, we choose the active form when possible. This amounts to 2α vertices.

By now, when a variable is assigned, then the variable reading gadgets corresponding to this variable have all of their u_a vertices already picked. Hence, all the cycles of this gadget are hit except for the negation gadget part. We then pick the vertex of the negation gadget that is shared with the clause gadget.

For a variable reading gadget of a variable that was not assigned we can just pick w_1 to hit all of the cycles of this gadget.

Regarding the clause gadgets, we know that our assignment satisfies all of the clauses so there should always be at least one literal that was assigned. Therefore, we must have picked a vertex of the clause gadget.

We picked W vertices to form our odd cycle transversal. $\triangleleft$

Claim 14 *If there is an odd cycle transversal of size W then the SAT instance can be satisfied.*

Proof Consider an odd cycle transversal T of size W . To hit the cycles of the variable choice gadgets, we need at least $r \sum_{j \in [k]} t_j$ vertices. To hit the arrows, we need at least 2α vertices. To hit the negation gadgets, we need at least ν vertices. Since this

amounts to W , we conclude that there is exactly one vertex per triangle in the variable choice gadgets, two vertices per arrow and one vertex per negation gadget.

We consider the assignment corresponding to the encoding stored in the variable choice gadgets. For each $X_{i,j}$, let b_a be the picked vertex of $T_{i,j}^a$. Then in our assignment we set the variable of $X_{i,j}$ with index $b_1 \dots b_{t_j}$ to true.

Since T is an odd cycle transversal, we know that for each clause, its corresponding cycle is hit. This means that for one of the negation gadgets, the chosen vertex is the clause gadget vertex. Since the odd cycles of the corresponding variable reading gadget are hit, it must be that all the arrow endpoints were picked, meaning that all these arrows are active, so the vertices that were picked in the variable choice gadget must be encoding the variable. We can conclude that our assignment satisfies the clause. $\triangleleft$

Let $t = \max_j t_j$. We have $t \leq \log_3(n) + 1 = \mathcal{O}(\log(n))$, where $n = |V|$.

Claim 15 $\text{pw}(G) \leq 6kt + 6$

Proof For each $i \in [r-1]$, the bags corresponding to $\phi(X_i, X_{i+1})$ contain the vertices of $\widehat{X_{i,j}}$ and $\widehat{X_{i+1,j}}$ for $j \in [k]$ (at most $6kt$). Then each clause gadget together with its connectors can be swept with 7 vertices at a time: we keep the first vertex of the cycle, the current vertex of the cycle, and the vertices w_1 and w_2 of the current variable reading gadget. The 3 remaining vertices suffice to sweep the arrows and the negation gadget. $\triangleleft$

The instances produced in the previous reduction are also instances of Feedback Vertex Set with similar budget. Indeed, removing an odd cycle transversal from arrows and negation gadgets, disconnects the gadgets and the gadgets do not contain even cycles once their odd cycles are broken. Hence the odd cycle transversals of the construction are also feedback vertex sets. Since a feedback vertex set must hit all cycles, it must hit at least the odd cycles, thus it is also an odd cycle transversal. $\square$

4.4 Problems Parameterized by Linear Mim-Width

In this section we show that several fundamental graph problems are XNLP-complete when parameterized by the linear mim-width of the input graph. For completeness, we state the concrete parameterized problems considered in this section.

INDEPENDENT SET / linear mim-width

Input: A graph $G = (V, E)$, a linear order of V of mim-width w , and an integer k .

Parameter: w .

Question: Does G contain an independent set of size at least k ?

DOMINATING SET / linear mim-width

Input: A graph $G = (V, E)$, a linear order of V of mim-width w , and an integer k .

Parameter: w .

Question: Is there a set $S \subseteq V$ of size at most k such that $S \cup N(S) = V$?

FEEDBACK VERTEX SET / linear mim-width

Input: A graph $G = (V, E)$, a linear order of V of mim-width w , and an integer k .

Parameter: w .

Question: Is there a set $S \subseteq V$ of size at most k such that $G - S$ is a forest?

 q -COLORING / linear mim-width

Input: A graph $G = (V, E)$, a linear order of V of mim-width w , and an integer k .

Parameter: w .

Question: Does G have a proper vertex-coloring with q colors?

XNLP-membership for these problems will be shown via the corresponding dynamic programming XP-algorithms [9, 20]. In all of these algorithms, the following equivalence relation is key to defining the table entries.

Definition 1 (*Neighborhood Equivalence*) Let G be a graph and $A \subseteq V(G)$. For all $X, Y \subseteq A$: $X \equiv_A Y \Leftrightarrow N(X) \cap (V(G) \setminus A) = N(Y) \cap (V(G) \setminus A)$.

Lemma 6 *The following problems parameterized by the mim-width of a linear order of the vertices of the input graph are in XNLP: INDEPENDENT SET, DOMINATING SET, q -COLORING for any fixed q , and FEEDBACK VERTEX SET.*

Proof In all cases, we will show membership using the respective dynamic programming XP-algorithms [9, 20]. In particular, we show that they can be turned into nondeterministic polynomial-time algorithms that uses at most (parameterized) logarithmic space. That is, if $v_1, \dots, v_n$ is the given linear order of the vertices of the input graph with mim-width w , each of the algorithms uses at most $O(w \log n)$ space.

With i going from 1 to n , at step i we store partial solutions associated with the subgraph of G induced by the vertices $V_i = \{v_1, \dots, v_i\}$. (For convenience, we let $\overline{V_i} = V \setminus V_i$.)

In all cases, partial solutions are indexed by integers whose values are upper bounded by n and by a constant number of vertex sets each of whose size is bounded by $O(w)$. In some cases, they are representatives of equivalence classes of $\equiv_{V_i}$, which is why we need the following claim, originally shown in [9]. We reprove it here to clarify that the procedure associated with it can be implemented in logarithmic space. $\triangleleft$

Claim 16 *For each $i \in [n - 1]$, and each $S_i \subseteq V_i$, there is a set $R_i \subseteq V_i$ with $R_i \equiv_{V_i} S_i$ and $|R_i| \leq w$. Furthermore, there is a polynomial-time algorithm using at most $O(w \log n)$ space that determines R_i from R_{i-1} , where $R_{i-1} \equiv_{V_{i-1}} S_i \cap V_{i-1}$ and $|R_{i-1}| \leq w$.*

Proof For $i \leq w$, we can simply let $R_i = S_i$, so suppose that $i > w \geq 1$, and that $|S_i| > w$. By induction, we can assume that we have $R_{i-1} \subseteq V_{i-1}$ of size at most w such that $R_{i-1} \equiv_{V_{i-1}} S_i \cap V_{i-1}$. Let $R'_i = R_{i-1} \cup \{v\}$. If $|R'_i| \leq w$, then we let $R_i = R'_i$ and we are done. We may assume that $|R'_i| = w + 1$. If there is some $x \in R'_i$ such that $N(R'_i \setminus \{x\}) \cap \overline{V_i} = N(R'_i) \cap \overline{V_i}$, then we let $R_i = R'_i \setminus \{x\}$ and we are done. Otherwise, we know that each vertex x in R'_i has a neighbor y in $\overline{V_i}$ such that y is non-adjacent to all vertices in $R'_i \setminus \{x\}$. This means that these xy -edges form an induced matching in $G[V_i, \overline{V_i}]$, a contradiction. $\triangleleft$

The algorithm then works as follows. Upon arrival of the next vertex v_{i+1} , we nondeterministically guess its interaction with the solution: in the cases of INDEPENDENT SET, DOMINATING SET, and FEEDBACK VERTEX SET, whether v_{i+1} is in the solution or not, and in the case of q -COLORING, which of the q colors v_{i+1} receives. We then nondeterministically guess the table index corresponding to the updated solution.

In each of the cases, the table entries consist of a collection of a constant number of vertex sets of size at most $O(w)$, which implies that the size of each table entry is bounded by $O(w \log n)$. For the cases when these sets are representatives of the neighborhood equivalence, we can use Claim 16 to conclude that the nondeterministic step can be implemented using only $O(w \log n)$ space and polynomial time as well.

Independent Set [9]. Here a table index consists of a single representative of an equivalence class of $\equiv_i$, and the table stores the size of a maximum independent set contained in the corresponding equivalence class. Such a table index requires only $O(w \log n)$ bits.

Dominating Set [9]. A table index consists of a pair of equivalence classes, $\mathcal{Q}_i$ of $\equiv_{V_i}$, and $\mathcal{R}_i$ of $\equiv_{\overline{V_i}}$. The table stores the minimum size of a set $Q \in \mathcal{Q}_i$ such that for any $R \in \mathcal{R}_i$, $Q \cup R$ dominates V_i . Since we only need two representatives, the table index requires again $O(w \log n)$ bits.

q-Coloring for fixed q [9]. Partial solutions are proper colorings of $G[V_i]$ and a table index consists of representatives of equivalence classes $\mathcal{Q}_1, \dots, \mathcal{Q}_q$ of $\equiv_{V_i}$ such that for all $i \in [q]$, color class i in the coloring is contained in $\mathcal{Q}_i$. Therefore the table index uses $O(qw \log n)$ bits, which is $O(w \log n)$ since q is a constant.

Feedback Vertex Set [20]. The algorithm from [20] solves the dual problem, MAXIMUM INDUCED FOREST. Here, partial solutions are induced forests F of the subgraph of G induced by V_i and all vertices from $\overline{V_i}$ that have a neighbor in V_i . The table indices consist of

- a forest R consisting (roughly speaking) of the restriction of F to $G[V_i, \overline{V_i}]$,
- a partition of the connected components of R , telling how they are joined together in F ,
- a minimal vertex cover S of $G[V_i, \overline{V_i}] - V(R)$, and
- the number of vertices of F .

The role of S is to control vertices that are either leaves of the solution F or internal vertices of F in V_i that have a neighbor in $\overline{V_i}$. In particular, it indicates that neither of these can have a neighbor in part of the solution coming from $\overline{V_i}$. In [20] it is shown that considering forests on at most $6w$ vertices as candidates for R suffices, which yields

that the first and second part of the table index can be represented using $O(w \log n)$ bits. The minimal vertex cover S may have many vertices, but it can be replaced by a representative of the equivalence class of $\equiv_{V_i}$ containing $S \cap V_i$, and a representative of the equivalence class of $\equiv_{\overline{V_i}}$ containing $S \cap \overline{V_i}$. Storing the size of F clearly only requires $O(\log n)$ bits. Therefore, we can have an equivalent definition of the table entries that again only use $O(w \log n)$ bits.

This finishes the proof of the lemma. $\square$

The following construction due to Fomin et al. [17] will be used in the reductions given in this section. It was used to prove W[1]-hardness of INDEPENDENT SET and DOMINATING SET on a parameterized family of graph classes which in turn implied W[1]-hardness of these problems parameterized by linear mim-width. For two disjoint sets $A, B \subseteq V$, the *bipartite complement* replaces each edge with one endpoint in A and the other in B with a non-edge and vice versa.

Definition 2 (Subdivision-complement, Fomin et al. [17]) Let $G = (V, E)$ be a graph, and $A, B \subseteq V$ with $A \cap B = \emptyset$. The *subdivision-complement between A and B* is the following operation:

1. Subdivide each edge uv with $u \in A$ and $v \in B$; call the resulting set of vertices R .
2. Take the bipartite complement between A and R and the bipartite complement between B and R .

The reason why this operation is useful for reductions for problems parameterized by mim-width are the following bounds on the maximum induced matching size of cuts resulting from this construction. This can also be derived from [17], but we include a simple direct proof here for completeness.

Lemma 7 Let $G = (V, E)$ be a graph, and $A, B \subseteq V$ with $A \cap B = \emptyset$. Let G' be the graph obtained from G by applying the subdivision-complement between A and B ; let R denote the set of vertices created in the construction. Then, for all $C \in \{A, B\}$, $\text{cutmim}_{G'}(C, R) \leq 2$.

Proof Suppose for a contradiction that there is an induced matching of size three in $G'[A, R]$, say $M = \{a_i r_i \mid i \in [3], a_i \in A, r_i \in R\}$. For all $i \in [3]$, let e_i denote the edge in G whose subdivision created vertex r_i . Since M is an induced matching and by construction, a_1 is the endpoint of e_2 and e_3 . But this implies that a_2 is not the endpoint of e_3 , and therefore that the edge $a_2 r_3$ exists in $G'[A, R]$. $\square$

To prove the bound on the mim-width of linear orders constructed in the hardness proofs in this section, we need the following additional lemma which can be seen as a variation of a lemma in [7], but for linear mim-width. Recall that for a graph $G = (V, E)$ and a partition $\mathcal{P}$ of V , the *quotient graph* $G/\mathcal{P}$ is the graph obtained from G by contracting each part of $\mathcal{P}$ into a single vertex. The *cutwidth* of a linear order $\Lambda = v_1, \dots, v_n$ of V , denoted by $\text{cutw}(\Lambda)$ is the maximum, over all i , of the number of edges with one endpoint in $\{v_1, \dots, v_i\}$ and the other in $\{v_{i+1}, \dots, v_n\}$.

Lemma 8 Let $G = (V, E)$ be a graph, let $\mathcal{P} = (P_1, \dots, P_r)$ be a partition of V , and let $G' = G/\mathcal{P}$. For all $i \in [r]$ let Λ_i be a linear order of P_i such that $\text{mimw}_{G[P_i]}(\Lambda_i) \leq$

c , and suppose that for all distinct $i, j \in [r]$, $\text{cutmim}_G(P_i, P_j) \leq d$. Let $\Lambda = \Lambda_1, \Lambda_2, \dots, \Lambda_r$, and let $\Lambda' = P_1, \dots, P_r$ be the corresponding linear order of $G/\mathcal{P}$. Then, $\text{mimw}(\Lambda) \leq 2d \cdot \text{cutw}(\Lambda') + c$.

Proof Let (A, B) be any cut induced by Λ , and let M be an induced matching in $G[A, B]$. Then, for some $i \in [r]$,

$$A = P_1 \cup \dots \cup P_{i-1} \cup (P_i \cap A), \text{ and } B = (P_i \cap B) \cup P_{i+1} \cup \dots \cup P_r.$$

The edges of M can now be split into the edges between $A \cap P_i$ and $B \cap P_i$, and for all $h \leq i$ and $j \geq i$, where at least one of the inequalities is strict, the edges between P_h and P_j . There are at most c edges of the first kind, since $\text{mimw}_{G[P_i]}(\Lambda_i) \leq c$, and Λ is equal to Λ_i on the vertices in P_i . For the second kind, for each pair P_h, P_j when $h < i$ and $j \geq i$, we have at most d edges, but only if $P_h P_j$ is an edge in $G/\mathcal{P}$. Therefore the number of such pairs is equal to the size of the cut between positions i_1 and i in the linear order Λ' , and therefore at most $\text{cutw}(\Lambda')$. Similarly, the number of pairs P_h, P_j with $h \leq i$ and $j > i$, that have edges in M is equal to the size of the cut between positions i and $i + 1$ in Λ' , and therefore again at most $\text{cutw}(\Lambda')$. Therefore the total number of the second kind of edges is at most $2d \cdot \text{cutw}(\Lambda')$. $\square$

Definition 3 (*Frame graph*) Let $(G = (V, E), V_1, \dots, V_r, f)$ be an instance of CHAINED MULTICOLORED CLIQUE or CHAINED MULTICOLORED INDEPENDENT SET; for each $i \in [r]$, let $V(i, 1), \dots, V(i, k)$ denote the partition of V_i according to f .

The frame graph $G' = (V', E')$ is obtained from G by applying, for each $h \in [r - 1]$ and each pair $(i_1, j_1), (i_2, j_2) \in \{h, h + 1\} \times [k]$, where $(i_1, j_1) <_{\text{LEX}} (i_2, j_2)$, the subdivision-complement between $V(i_1, j_1)$ and $V(i_2, j_2)$.³ We denote the set of new vertices by $R(i_1, j_1, i_2, j_2)$.

For convenience, we let $\mathcal{P}$ denote the partition of V' into $V(1, 1), \dots, V(r, k)$, $R(1, 1, 1, 1), \dots, R(r, k, r, k)$, and we define the following auxiliary partial function $\phi: E \rightarrow V'$: For all (i_1, j_1) and (i_2, j_2) as above, for each $v_1 \in V(i_1, j_1)$ and $v_2 \in V(i_2, j_2)$ with $v_1 v_2 \in E$, we let $\phi(v_1 v_2) \in R(i_1, j_1, i_2, j_2)$ be the vertex created when subdividing $v_1 v_2$.

The following argument will be repeated in several proofs, we therefore extract it as a separate lemma.

Lemma 9 Let $(G = (V, E), V_1, \dots, V_r, f)$ be an instance of CHAINED MULTICOLORED CLIQUE, and let $G' = (V', E')$ be its frame graph; adapt the notation from Definition 3. Then, G' has an independent set S with $|S \cap P| = 1$ for all $P \in \mathcal{P}$ if and only if G has a chained multicolored clique.

Proof Suppose G' has an independent set S with $|S \cap P| = 1$ for all $P \in \mathcal{P}$. Let $v_{i,j} \in S \cap V(i, j)$ for all $i \in [r]$, $j \in [k]$. We claim that this implies that for all $h \in [r - 1]$, and all $(i_1, j_1), (i_2, j_2) \in \{h, h + 1\} \times [k]$ with $(i_1, j_1) <_{\text{LEX}} (i_2, j_2)$,

³ Here, $<_{\text{LEX}}$ denotes the lexicographic ordering. We have $(i_1, j_1) <_{\text{LEX}} (i_2, j_2)$ if either $i_1 < i_2$ or if $i_1 = i_2$ and $j_1 < j_2$.

we have that $\phi(v_{i_1, j_1} v_{i_2, j_2}) \in S \cap R(i_1, j_1, i_2, j_2)$, which implies that $v_{i_1, j_1} v_{i_2, j_2} \in E$ and in particular that $S \cap V$ is a chained multicolored clique in G .

Let $r \in S \cap R(i_1, j_1, i_2, j_2)$ and suppose $r \neq \phi(v_{i_1, j_1} v_{i_2, j_2})$. We may assume that $r = \phi(v, w)$ where $v \in V(i_1, j_1) \setminus \{v_{i_1, j_1}\}$. But then, $v_{i_1, j_1} r$ is an edge in G' , a contradiction.

For the other direction, let $W \subseteq V$ be the chained multicolored clique in G . Let $S = \emptyset$. For each $i \in [r]$ and $j \in [k]$, we add the vertex $v_{i, j} \in W \cap V(i, j)$ to S . Next, for each $h \in [r - 1]$, and each pair $(i_1, j_1), (i_2, j_2) \in \{h, h + 1\} \times [k]$ where $(i_1, j_1) <_{LEX} (i_2, j_2)$, we add $\phi(v_{i_1, j_1} v_{i_2, j_2})$ to S . Note that since W is a chained multicolored clique, the edge $v_{i_1, j_1} v_{i_2, j_2}$ always exists in G .

It follows immediately from the construction that S is an independent set in G' , and that for all $P \in \mathcal{P}$, $|S \cap P| = 1$. $\square$

Lemma 10 INDEPENDENT SET parameterized by the mim-width of a given linear order of the input graph is XNLP-hard.

Proof We give a parameterized logspace reduction from CHAINED MULTICOLORED CLIQUE (CMC); let $\mathcal{I} = (G = (V, E), V_1, \dots, V_r, f)$ be an instance of CMC.

We create an INDEPENDENT SET instance whose graph $G' = (V', E')$ is the frame graph of $\mathcal{I}$ (Definition 3). We adapt the notation from Definition 3. We make each $P \in \mathcal{P}$ a clique in G' , and we let $k' = |\mathcal{P}| = (r - 1)2k^2 + k$.

Since G' is simply the frame graph of $\mathcal{I}$ where each P is turned into a clique, correctness of this reduction follows immediately from Lemma 9.

Claim 17 One can in polynomial time and logarithmic space construct a linear order Λ of V' such that $\text{mimw}(\Lambda) = O(k^2)$.

Proof For all $i \in [r]$ and $j \in [k]$, we let $\Lambda(i, j)$ be an arbitrary linear order of $V(i, j)$. For all $h \in [r - 1]$, and all $(i_1, j_1), (i_2, j_2) \in \{h, h + 1\} \times [k]$ with $(i_1, j_1) <_{LEX} (i_2, j_2)$, we let $\Gamma(i_1, j_1, i_2, j_2)$ be an arbitrary linear order of $R(i_1, j_1, i_2, j_2)$. The desired linear order Λ traverses V' as follows: Consider $(i, j) \in [r] \times [k]$ in lexicographically increasing order. First, we follow $\Lambda(i, j)$, and then $\Gamma(i, j, i, j + 1), \dots, \Gamma(i, j, i, k)$, and if $i < r$, then $\Gamma(i, j, i + 1, 1), \dots, \Gamma(i, j, i + 1, k)$. It is clear that this linear order of G' can be created using $O(\log n)$ bits of memory, where n is the number of vertices of G .

Clearly, each $\Lambda(i, j)$ and each $\Gamma(i_1, j_1, i_2, j_2)$ has mim-width at most 1. The only edges in G' between different parts of $\mathcal{P}$ are between $V(i_1, j_1)$ and $R(i_1, j_1, i_2, j_2)$, and between $V(i_2, j_2)$ and $R(i_1, j_1, i_2, j_2)$, where $(i_1, j_1) <_{LEX} (i_2, j_2)$. By construction it therefore follows from Lemma 7 that for each pair of distinct parts $P_1, P_2 \in \mathcal{P}$, $\text{cutmim}_{G'}(P_1, P_2) \leq 2$.

Let Λ' is the linear order of the vertices of $G'/\mathcal{P}$ where the vertices of $G/\mathcal{P}$ appear in the same order as in Λ . We can observe that $\text{cutw}(\Lambda') = O(k^2)$, and therefore the claim follows from Lemma 8. $\triangleleft$

This concludes the proof of Lemma 10. $\square$

Lemma 11 DOMINATING SET parameterized by the mim-width of a given linear order of the input graph is XNLP-hard.

Proof The proof is very similar to that of Lemma 10, so we mainly point out the differences; the first one is that we reduce from CHAINED MULTICOLORED INDEPENDENT SET instead of CHAINED MULTICOLORED CLIQUE. Let $\mathcal{I} = (G, V_1, \dots, V_r, f)$ be an instance of CHAINED MULTICOLORED INDEPENDENT SET.

We obtain the graph G'' of the DOMINATING SET instance in two steps. We first create the frame graph G' of $\mathcal{I}$, adapt the notation from Definition 3, and make each $P \in \mathcal{P}$ a clique in G' . To obtain G'' from G' , we add a vertex $z_{i,j}$ whose neighborhood is $V(i, j)$, for each $(i, j) \in [r] \times [k]$; we let $V(i, j)'' = V(i, j) \cup \{z_{i,j}\}$, and $Z = \{z_{i,j} \mid i \in [r], j \in [k]\}$. We let $k'' = rk$.

Suppose that G has a chained multicolored independent set W . We claim that W is a dominating set in G'' , and clearly $|W| = rk = k''$. Since each $V(i, j)''$ is a clique, and $W \cap V(i, j) \neq \emptyset$, we have that for all $v \in V(i, j)''$, $N_{G'}[v] \cap W \geq 1$. Next suppose that there is some $r \in R(i_1, j_1, i_2, j_2)$ with $N(r) \cap W = \emptyset$. Let e denote the edge in G corresponding to r . The only non-neighbor of r in $V(i_1, j_1)$ is the endpoint v_1 of e in $V(i_1, j_1)$, and the only non-neighbor of r in $V(i_2, j_2)$ is the endpoint v_2 of e in $V(i_2, j_2)$. This implies that $\{v_1, v_2\} \subseteq W$, but then e is an edge between two vertices in W in G , a contradiction.

Now suppose that G'' has a dominating set S of size k'' . Due to the vertices in Z , we may assume that each $V(i, j)$ contains a vertex from S . Using similar arguments as in the previous paragraph, we can conclude that S is a chained multicolored independent set in G . Using the same construction as in Claim 17, only taking into account the vertices in Z , we can argue that we can construct a linear order of G'' with mim-width at most $O(k^2)$ in polynomial time and logarithmic space. $\square$

Lemma 12 FEEDBACK VERTEX SET parameterized by the mim-width of a given linear order of the vertices of the input graph is XNLP-hard.

Proof Again the proof is very similar to that of Lemma 10. We give a parameterized logspace reduction from CHAINED MULTICOLORED CLIQUE to the dual problem of FEEDBACK VERTEX SET, MAXIMUM INDUCED FOREST. Let $\mathcal{I} = (G, V_1, \dots, V_r, f)$ be an instance of CHAINED MULTICOLORED CLIQUE. We first construct $G' = (V', E')$ as the frame graph of $\mathcal{I}$, adapting the notation of Definition 3, and making each $P \in \mathcal{P}$ a clique in G' . The graph G'' of the MAXIMUM INDUCED FOREST instance is then obtained from G' by adding

- for each $P \in \mathcal{P}$, two vertices a_P, b_P with $N(a_P) = N(b_P) = P$, and
- one more vertex c with $N(c) = V'$.

We let $P'' = P \cup \{a_P, b_P\}$, $A = \{a_P \mid P \in \mathcal{P}\}$, $B = \{b_P \mid P \in \mathcal{P}\}$, and $Z = A \cup B \cup \{c\}$.

We let $k'' = 3|\mathcal{P}| + 1 = 6k^2(r - 1) + 3k + 1$.

Suppose that G has a chained multicolored clique W . Let S be an independent set in G' with $|S \cap P| = 1$ for all $P \in \mathcal{P}$ which exists by Lemma 9. We let $S'' = S \cup A \cup B \cup \{c\}$. Since S is also an independent set in G'' , it follows from the construction that S'' induces a tree on k'' vertices in G'' .

Suppose that G'' has an induced forest S on k'' vertices. Since each $P \in \mathcal{P}$ is a clique, $|S \cap P| \leq 2$. Moreover, the constraint imposed by k'' requires us to use three vertices from each P'' ; and if $|S \cap P| = 2$, we have that $a_P \cup (S \cap P)$ and $b_P \cup (S \cap P)$

are triangles. We therefore have that $|S \cap P| = 1$ and that $A \cup B \subseteq S$. Moreover, k'' demands that c is in S as well. Now, the only way for S to induce a forest in G'' is if $S \cap \bigcup_{P \in \mathcal{P}} P$ is an independent set of size $|\mathcal{P}|$ in G' . We can conclude by Lemma 9 that this implies a multicolored clique in G .

The polynomial time and logarithmic space construction of a linear order of mim-width at most $O(k^2)$ of G'' can once more be done in analogy with Claim 17. $\square$

Lemma 13 *For any fixed $q \geq 5$, q -COLORING parameterized by the mim-width of a given linear order of the vertices of the input graph is XNLP-hard.*

Proof We give a parameterized logspace reduction from CHAINED MULTICOLORED CLIQUE to 5-LIST-COLORING. Let $\mathcal{I} = (G = (V, E), V_1, \dots, V_r, f)$ be the instance of CHAINED MULTICOLORED CLIQUE.

We create the graph $G'' = (V'', E'')$ of the 5-LIST-COLORING instance as follows:

Let $G' = (V', E')$ be the frame graph of $\mathcal{I}$ and adapt the notation of Definition 3. We obtain G'' and the lists $L'' = \{L(v) \mid v \in V''\}$ as follows:

- For each $P \in \mathcal{P}$, we add two vertices $a(P)$ and $b(P)$, and make $P'' = P \cup \{a(P), b(P)\}$ a path from $a(P)$ to $b(P)$. We let $\mathcal{P}'' = \{P'' \mid P \in \mathcal{P}\}$.
- For each $(i, j) \in [r] \times [k]$, each list of a vertex in $P = V(i, j)$ is $[3]$. If $|P|$ is even, the lists of both $a(P)$ and $b(P)$ are $\{1\}$; and if $|P|$ is odd, the list of $a(P)$ is $\{1\}$, and the list of $b(P)$ is $\{2\}$.
- For each $h \in [r - 1]$ and $(i_1, j_1), (i_2, j_2) \in \{h, h + 1\} \times [k]$ with $(i_1, j_1) <_{LEX} (i_2, j_2)$, each list of a vertex in $R = R(i_1, j_1, i_2, j_2)$ is $\{3, 4, 5\}$. If $|R|$ is even, the lists of both $a(R)$ and $b(R)$ are $\{5\}$; and if $|R|$ is odd, the list of $a(R)$ is $\{5\}$, and the list of $b(R)$ is $\{4\}$.

The following observation is immediate from the above construction.

Observation 2 *In each proper list coloring of (G'', L'') and each $P \in \mathcal{P}$, there is a vertex in P that received color 3. Conversely, if some vertex $v \in P$ received color 3 in a proper list coloring of (G'', L'') , then the vertices in $P'' \setminus \{v\}$ can be properly list-colored with colors $\{1, 2\}$, if $P = V(i, j)$ for some i, j or with colors $\{4, 5\}$, if $P = R(i_1, j_1, i_2, j_2)$, for some i_1, i_2, j_1, j_2 .*

Now suppose that G has a chained multicolored clique W . Then by Lemma 9, there is an independent set S in G' such that $|S \cap P| = 1$ for all $P \in \mathcal{P}$. Note that S is also an independent set in G'' . We can therefore let S be color class 3, and by Observation 2, the remaining vertices of each path P can be properly list colored without using color 3. It suffices to check the edges between $V(i_1, j_1)$ and $R(i_1, j_1, i_2, j_2)$, for any valid choice of i_1, i_2, j_1, j_2 .

Since S is an independent set, the vertices $v \in S \cap V(i_1, j_1)$ and $w \in S \cap R(i_1, j_1, i_2, j_2)$ are non-adjacent. Furthermore, v has a different color from all vertices in $R(i_1, j_1, i_2, j_2) \setminus S$. Finally, the sets of colors appearing on $V(i_1, j_1) \setminus S$ and $R(i_1, j_1, i_2, j_2) \setminus S$ are disjoint.

Conversely, suppose that (G'', L'') has a proper list-coloring. Then we can combine Observation 2 and Lemma 9 to conclude that G has a chained multicolored clique (with color class 3 being the independent set required by Lemma 9).

It remains to argue that we can create a linear order of G'' of mim-width at most $O(k^2)$ in polynomial time and logarithmic space.

This can once more be done in analogy with Claim 17, with the linear order on each $P'' \in \mathcal{P}''$ now following the path from one endpoint to the other (which has mim-width 1).

We have shown that 5-LIST-COLORING parameterized by the mim-width of a given linear order of the input graph is XNLP-hard. To derive XNLP-hardness of 5-COLORING in the same parameterization, observe that we can use the standard trick of adding a clique on vertices $\{1, \dots, 5\}$, and for each $i \in [5]$, connecting i and v if $i \notin L(v)$. Since adding c vertices can only increase the mim-width of a given linear order by at most c , no matter where the new vertices are placed, this does not prohibitively increase the linear mim-width either.

To obtain hardness for any $q > 5$, we simply add $q - 5$ universal vertices to the 5-COLORING instance obtained in the previous paragraph.

We observe that this does not increase the linear mim-width (and that we can trivially obtain a linear order for the graph with the added universal vertices): Let G be any graph with at least one edge (which implies that any of its linear orders has mim-width at least one), and let G' be obtained from G by adding a universal vertex v . Let Λ be any linear order on the vertices of G , and let Λ' be obtained from Λ by adding v as the last vertex in the order. In any cut induced by Λ' , if v is part of an induced matching, then this matching necessarily has size one, since v is universal. Therefore, $\text{mimw}(\Lambda') = \text{mimw}(\Lambda)$. $\square$

Combining Lemmas 6 and 10 to 13, we obtain the main result of this section.

Theorem 7 *When parameterized by linear mim-width, INDEPENDENT SET, DOMINATING SET, q -COLORING for any fixed $q \geq 5$ and FEEDBACK VERTEX SET are XNLP-complete.*

4.5 Bipartite Bandwidth

A *caterpillar* is a tree where all vertices of degree at least three are on a common path. Bodlaender et al. [4] showed that the following problem is XNLP-complete.

BANDWIDTH ON CATERPILLARS

Input: A caterpillar $G = (V, E)$ and an integer k .

Parameter: k .

Question: Is there a bijection $f : V \rightarrow [1, |V|]$ such that for all edges $\{v, w\} \in E$: $|f(v) - f(w)| \leq k$?

We reduce from that problem in order to prove the following result.

Theorem 8 *BIPARTITE BANDWIDTH is XNLP-complete for trees.*

Proof We first show membership. Consider a bipartite graph $G = (X, Y, E)$ and integer k . We can assume $|X| = |Y|$ since adding isolated vertices does not increase the bipartite bandwidth. We will maintain the following certificate: the $2k + 1$ last chosen vertices of X and the $2k + 1$ last chosen vertices of Y . This takes $(2k + 1) \log n$

bits. For $i = 1, \dots, |X|$, we guess the i th vertex of x_i of in the order α and the i th vertex y_i in the order β . If $i \geq 2k + 1$, we verify that all neighbors of x_{i-k} are among $\{y_{i-2k}, \dots, y_i\}$ and that all neighbors of y_{i-k} are among $\{x_{i-2k}, \dots, x_i\}$.

We show hardness via a reduction from BANDWIDTH ON CATERPILLARS. Take a caterpillar $G = (V, E)$ and integer k . Set $L = k^4 + 2k^2$.

Build a graph H_0 as follows. For each $v \in V$, take a vertex x_v in H_0 with $2k^4 = 2L - 4k^2$ vertices of degree 1 adjacent to it. Replace each edge $\{v, w\} \in E$ by a path with $2k$ edges.

Now, take an arbitrary vertex $v \in V$. Take two copies of H_0 , and add an edge between the two copies of x_v , for this vertex v . Let H be the resulting graph.

Claim 18 *If H has bipartite bandwidth at most L , then G has bandwidth at most k .*

Proof Fix a copy of H_0 . All vertices of the form x_v in this copy are in the same side of bipartition (X, Y) of H , say all are in X . Suppose (α, β) are the orderings of (X, Y) that show that H has bipartite bandwidth at most L . Let g be the layout of G where $g(v) < g(w)$ iff $\alpha(x_v) < \alpha(x_w)$. We claim this ordering has bandwidth at most k .

Look at v and w with $g(w) = g(v) + 1$. There are $4k^4$ leaves adjacent to x_v and x_w and for any such leaf ℓ , we know that $\beta(\ell) \in [\alpha(x_v) - L, \alpha(x_w) + L]$. This implies that $\alpha(x_w) - \alpha(x_v) \geq 4k^4 - (2L + 1)$.

Consider an edge $\{v, w\} \in E$ and suppose w.l.o.g. that v comes before w in g . Repeatedly applying the observation above, if $g(w) = g(v) + q$, then

$$\alpha(x_w) \geq \alpha(x_v) + 2q(2k^4 - L) - q = \alpha(x_v) + q(2k^4 - 4k^2 - 1).$$

We may assume that k is large enough so that $2k^4 - 4k^2 - 1 > L = k^4 + 2k^2$. This implies that when $g(w) > g(v) + k$, then $\alpha(x_w) > \alpha(x_v) + 2kL$. But there is also a path $x_v = u_0, u_1, \dots, u_{2k} = x_w$ with $2k$ edges between x_v and x_w , and we find that

$$|\alpha(u_{2k}) - \alpha(u_0)| \leq \sum_{i=0}^{k-1} |\alpha(u_{2i}) - \beta(u_{2i+1})| + |\beta(u_{2i+1}) - \alpha(u_{2i+2})| \leq 2kL.$$

This gives a contradiction, so we must have $g(w) \leq g(v) + k$ for all edges $\{v, w\}$. It follows that G has bandwidth at most k . $\triangleleft$

Claim 19 *If G has bandwidth at most k , then H_0 has bandwidth at most L .*

Proof We build an ordering on the vertices of H_0 as follows. First, we order the vertices of the form x_v in the same order as the vertices of the form v in the layout of G . For each x_v , insert half of its incident leaves directly before x_v in the ordering, and half of its incident leaves directly after x_v .

For each edge $\{v, w\} \in E$, we have to insert the vertices of the path with $2k - 1$ vertices between x_v and x_w . Suppose v is earlier in the ordering than w , with $y_1, y_2, \dots, y_\ell$ between v and w , in this order, with $\ell \leq k - 1$. Write $y_{\ell+1} = w$. Insert the $2k - 1$ vertices on the path consecutively in the ordering as follows: directly before the first incident leaf of x_{y_1} , directly before x_{y_1} , directly before the first incident leaf of x_{y_2} , directly before x_{y_2} , etc., until we placed a vertex of the path directly before

the first leaf of $x_{y_{\ell+1}} = x_w$. If the path has remaining vertices left, split these evenly before x_w and before its first leaf.

Let a ‘slot’ be defined as a vertex x_v along with all vertices added before it by the procedure above, or the set of vertices added right before a leaf of a vertex x_v . We now show that all slots have size at most $k(k-1)+1$. We first count the vertices added due to an edge uv . For each $i \in [k]$, there is at most one such u at distance i from v in the layout, and this contributes $k-i+1$ vertices before the first leaf of x_v , and $k-i$ vertices before x_v . We now count the vertices due to edges uw from a vertex u strictly before v to a vertex w strictly after v . We count one vertex for each such edge. If u is the i th vertex before v , for $i \in [k-1]$, then there will be at most $k-i$ such edges coming from u . The number of vertices in the slot of x_v is hence at most $\left(\sum_{i=1}^k k-i\right) + \left(\sum_{i=1}^{k-1} k-i\right) = 1+k(k-1)$, whereas the number of vertices in the slot before the first leaf of x_v is at most $\left(\sum_{i=1}^k k-i+1\right) + \left(\sum_{i=1}^{k-1} k-i\right) = k^2$.

If $k > 0$, then $1+k(k-1) \leq k^2$, so all slots have at most k^2 vertices. Edges of H_0 are either on a path between vertices x_v, x_w , or edges from x_v to a leaf. The number of vertices in between the endpoints of the edge in the order is hence at most k^4 (allowing leaves to be in between) plus k^2 (allowing a ‘slot’ to be in between). So we find the endpoints of an edge of H_0 are at most $k^4 + k^2 + 1 \leq L$ apart. We conclude that H_0 has bandwidth at most L .

If $k = 0$, G has no edge and H_0 has no edge so it has bandwidth $L = 0$. $\triangleleft$

Claim 20 *If H_0 has bandwidth at most L , then H has bipartite bandwidth at most L .*

Proof Suppose f is a layout of H_0 of bandwidth at most k . Let (X, Y) be the bipartition of H_0 . We set $\alpha(x) = f(x)$ for all $x \in X$ and $\beta(y) = f(y)$ for all $y \in Y$. For a copy x' of the vertex $x \in X$ in the second copy of H , we set $\beta(x') = f(x)$, and similarly we set $\alpha(y') = f(y)$ for a copy y' of a vertex y . We find that $|\alpha(x) - \beta(y)| = |f(x) - f(y)| \leq L$ whenever $\{x, y\}$ is an edge of H_0 , and similarly $|\alpha(y') - \beta(x')| \leq L$. There is only one edge $\{x, x'\}$ between vertices of the two copies, and $\alpha(x) = \beta(x')$. $\triangleleft$

The three claims show that H has bipartite bandwidth at most L if and only if G has bandwidth at most k , as desired. $\square$

5 Conclusion

In this paper, we gave a number of XNLP-completeness proofs for graph problems parameterized by linear width measures. Such results are interesting for a number of reasons: they pinpoint the “right” complexity class for parameterized problems, they imply hardness for all classes $W[t]$, and they tell that it is unlikely that there is an algorithm that uses ‘XP time and FPT space’ by Conjecture 1.

This paper gives among others the first examples of XNLP-complete problems when the linear clique-width or linear mim-width is taken as parameter. Our hardness results give new starting points for future hardness and completeness proofs, in particular for problems with width measures like pathwidth, (linear) clique-width, or (linear) mim-width as parameter.

The class of XNLP is targeted at problems with a linear structure. Very recently, Bodlaender et al. [2] introduced the class XALP, which appears to capture the complexity of problems with a tree-like structure, e.g., LIST COLORING with treewidth as parameter is XALP-complete. It would be interesting to verify if our results can be modified to XALP-completeness proofs when we use the ‘tree-variants’ of the respective width parameters (e.g., clique-width instead of linear clique-width, etc.).

Other interesting directions for future research in this line are, for instance, to consider the parameterization by *cutwidth*; a promising candidate problem to show XNLP-completeness parametrized by cutwidth is LIST EDGE COLORING. Another interesting parameter to consider in this context is the *degeneracy* of a graph. It is also interesting to explore the concept of XNLP-completeness for width measures of other objects than graphs. One could for instance consider (linear) width measures of digraphs or hypergraphs.

We also leave open what the correct parameterized complexity class is for FEEDBACK VERTEX SET parameterized by logarithmic pathwidth or logarithmic linear clique-width – we showed XNLP-hardness, but did not prove containment in XNLP.

Acknowledgements Carla Groenland was supported by the European Union’s Horizon 2020 research and innovation programme under the ERC grant CRACKNP (number 853234) and the Marie Skłodowska-Curie grant GRAPHCOSY (number 101063180). Lars Jaffke was supported by the Norwegian Research Council (project number 274526) and the Meltzer Research Fund. The authors have no further relevant financial or non-financial interests to disclose.

Author Contributions All authors contributed equally to all stages of the research that led to this paper.

Funding Open access funding provided by Norwegian School of Economics.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Bodlaender, H.L., Cornelissen, G., Wegen, M.: Problems hard for treewidth but easy for stable gonality. In: Bekos, M.A., Kaufmann, M. (eds.) Proceedings 48th international workshop on graph-theoretic concepts in computer science (WG 2022). Lecture notes in computer science, vol. 13453, pp. 84–97 (2022). https://doi.org/10.1007/978-3-031-15914-5_7
2. Bodlaender, H.L., Groenland, C., Jacob, H., Pilipczuk, M., Pilipczuk, M.: On the complexity of problems on tree-structured graphs. In: Dell, H., Nederlof, J. (eds.) 17th international symposium on parameterized and exact computation (IPEC 2022). Leibniz international proceedings in informatics (LIPIcs), vol. 249, pp. 6–1617. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2022). <https://doi.org/10.4230/LIPIcs.IPEC.2022.6> . <https://drops.dagstuhl.de/opus/volltexte/2022/17362>

3. Bodlaender, H.L., Groenland, C., Jacob, H., Jaffke, L., Lima, P.T.: XNLP-Completeness for parameterized problems on graphs with a linear structure. In: Dell, H., Nederlof, J. (eds.) 17th international symposium on parameterized and exact computation (IPEC 2022). Leibniz international proceedings in informatics (LIPIcs), vol. 249, pp. 8:1–8:18. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany (2022). <https://doi.org/10.4230/LIPIcs.IPEC.2022.8>
4. Bodlaender, H.L., Groenland, C., Nederlof, J., Swennenhuis, C.M.F.: Parameterized problems complete for nondeterministic FPT time and logarithmic space. In: Proceedings 62nd IEEE annual symposium on foundations of computer science, FOCS 2021, pp. 193–204 (2021). <https://doi.org/10.1109/FOCS52979.2021.00027>
5. Bodlaender, H.L., Lokshtanov, D., Penninkx, E.: Planar capacitated dominating set is W[1]-hard. In: Chen, J., Fomin, F.V. (eds.) Proceedings 4th international workshop on parameterized and exact computation, IWPEC 2009. Lecture notes in computer science, vol. 5917, pp. 50–60. Springer (2009). https://doi.org/10.1007/978-3-642-11269-0_4
6. Bodlaender, H.L.: Parameterized complexity of bandwidth of caterpillars and weighted path emulation. In: Kowalik, L., Pilipczuk, M., Rzazewski, P. (eds.) Proceedings of the 47th international workshop on graph-theoretic concepts in computer science (WG 2021). Lecture notes in computer science, vol. 12911, pp. 15–27. Springer (2021). https://doi.org/10.1007/978-3-030-86838-3_2
7. Brettell, N., Horsfield, J., Munaro, A., Paulusma, D.: List k -colouring P_t -free graphs: a mim-width perspective. Inf. Process. Lett. **173**, 106168 (2022). <https://doi.org/10.1016/j.ipl.2021.106168>
8. Broersma, H., Golovach, P.A., Patel, V.: Tight complexity bounds for FPT subgraph problems parameterized by the clique-width. Theor. Comput. Sci. **485**, 69–84 (2013). <https://doi.org/10.1016/j.tcs.2013.03.008>
9. Bui-Xuan, B., Telle, J.A., Vatshelle, M.: Fast dynamic programming for locally checkable vertex subset and vertex partitioning problems. Theor. Comput. Sci. **511**, 66–76 (2013). <https://doi.org/10.1016/j.tcs.2013.01.009>
10. Cygan, M., Fomin, F.V., Kowalik, L., Lokshtanov, D., Marx, D., Pilipczuk, M., Pilipczuk, M., Saurabh, S.: Parameterized algorithms. Springer, Cham (2015). <https://doi.org/10.1007/978-3-319-21275-3>
11. Dom, M., Lokshtanov, D., Saurabh, S., Villanger, Y.: Capacitated domination and covering: a parameterized perspective. In: Grohe, M., Niedermeier, R. (eds.) Proceedings 3rd international workshop on parameterized and exact computation, IWPEC 2008. Lecture notes in computer science, vol. 5018, pp. 78–90. Springer (2008). https://doi.org/10.1007/978-3-540-79723-4_9
12. Downey, R.G., Fellows, M.R.: Parameterized complexity. Springer, New York (1999)
13. Elberfeld, M., Stockhusen, C., Tantau, T.: On the space and circuit complexity of parameterized problems: classes and completeness. Algorithmica **71**(3), 661–701 (2015). <https://doi.org/10.1007/s00453-014-9944-y>
14. Fomin, F.V., Golovach, P.A., Lokshtanov, D., Saurabh, S.: Intractability of clique-width parameterizations. SIAM J. Comput. **39**(5), 1941–1956 (2010). <https://doi.org/10.1137/080742270>
15. Fomin, F.V., Golovach, P.A., Lokshtanov, D., Saurabh, S.: Almost optimal lower bounds for problems parameterized by clique-width. SIAM J. Comput. **43**(5), 1541–1563 (2014). <https://doi.org/10.1137/130910932>
16. Fomin, F.V., Golovach, P.A., Lokshtanov, D., Saurabh, S., Zehavi, M.: Clique-width III: Hamiltonian cycle and the odd case of graph coloring. ACM Trans. Algorithms **15**(1), 9–1927 (2019). <https://doi.org/10.1145/3280824>
17. Fomin, F.V., Golovach, P.A., Raymond, J.: On the tractability of optimization problems on H-graphs. Algorithmica **82**(9), 2432–2473 (2020). <https://doi.org/10.1007/s00453-020-00692-9>
18. Guillemot, S.: Parameterized complexity and approximability of the longest compatible sequence problem. Discret. Optim. **8**(1), 50–60 (2011). <https://doi.org/10.1016/j.disopt.2010.08.003>
19. Gurski, F., Wanke, E.: Line graphs of bounded clique-width. Discret. Math. **307**(22), 2734–2754 (2007). <https://doi.org/10.1016/j.disc.2007.01.020>
20. Jaffke, L., Kwon, O.-J., Telle, J.A.: Mim-width. II The feedback vertex set problem. Algorithmica **82**, 118–145 (2020)
21. Johansson, O.: Graph decompositions using node labels. PhD thesis, KTH Stockholm, Sweden (2001)
22. Kobler, D., Rotics, U.: Edge dominating set and colorings on graphs with fixed clique-width. Discret. Appl. Math. **126**(2–3), 197–221 (2003). [https://doi.org/10.1016/S0166-218X\(02\)00198-1](https://doi.org/10.1016/S0166-218X(02)00198-1)
23. Lokshtanov, D., Marx, D., Saurabh, S.: Known algorithms on graphs of bounded treewidth are probably optimal. ACM Trans. Algorithms **14**(2), 13–11330 (2018). <https://doi.org/10.1145/3170442>

24. Mathieson, L., Szeider, S.: The parameterized complexity of regular subgraph problems and generalizations. In: Harland, J., Manyem, P. (eds.) Proceedings 14th computing: the Australasian theory symposium, CATS 2008. CRPIT, vol. 77, pp. 79–86. Australian Computer Society (2008). <http://crpit.scem.westernsydney.edu.au/abstracts/CRPITV77Mathieson.html>
25. Moser, H., Thilikos, D.M.: Parameterized complexity of finding regular induced subgraphs. *J. Discret. Algorithms* 7(2), 181–190 (2009). <https://doi.org/10.1016/j.jda.2008.09.005>
26. Pilipczuk, M., Wrochna, M.: On space efficiency of algorithms working on structural decompositions of graphs. *ACM Trans. Comput. Theory* 9(4), 1–36 (2018). <https://doi.org/10.1145/3154856>
27. Vatshelle, M.: New width parameters of graphs. PhD thesis, University of Bergen, Norway (2012)
28. Wanke, E.: k -NLC graphs and polynomial algorithms. *Discret. Appl. Math.* 54(2–3), 251–266 (1994). [https://doi.org/10.1016/0166-218X\(94\)90026-4](https://doi.org/10.1016/0166-218X(94)90026-4)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.